Contents

English | |

String Operation Function Reference

A list of operation functions for strings (`str`). All functions support UFCS notation.

> **Note:** All string operations are UTF-8 aware. `len()`, `char_at()`, `substring()`, `find()`, and `reverse()` operate on Unicode code points, not bytes. Use `byte_len()` if you need the byte length.

Function List

Search and Check

Function	Signature	Description
`contains`	`(str, str) -> bool`	Returns whether a substring is contained
`starts_with`	`(str, str) -> bool`	Returns whether it starts with a prefix

Function	Signature	Description
`ends_with`	`(str, str) -> bool`	Returns whether it ends with a suffix
`find`	`(str, str) -> Option<int>`	Returns the character position of a substring (`None` if not found)

Extraction and Transformation

Function	Signature	Description
`substring`	`(str, int, int) -> str`	Extract a substring (character indices)
`char_at`	`(str, int) -> str`	Get the UTF-8 character at a specified position
`replace`	`(str, str, str) -> str`	Replace all occurrences of a substring

Case Conversion

Function	Signature	Description
`to_upper`	`str -> str`	Convert to ASCII uppercase
`to_lower`	`str -> str`	Convert to ASCII lowercase

Whitespace Removal

Function	Signature	Description
`trim`	`str -> str`	Remove leading and trailing whitespace
`trim_start`	`str -> str`	Remove leading whitespace
`trim_end`	`str -> str`	Remove trailing whitespace

Generation and Processing

Function	Signature	Description
`repeat`	`(str, int) -> str`	Repeat a string n times
`reverse`	`str -> str`	Reverse a string (UTF-8 aware)
`byte_len`	`str -> int`	Returns the byte length of a string

Split and Join

Function	Signature	Description
`split`	`(str, str) -> List<str>`	Split by delimiter
`join`	`(List<str>, str) -> str`	Join with separator

Type Conversion

Function	Signature	Description
`to_int`	`str -> int`	Convert string to integer
`to_float`	`str -> float`	Convert string to floating-point number
`to_str`	`int/float/bool/str -> str`	Convert value to string

---

contains

Signature: `contains(s: str, sub: str) -> bool`

Returns whether string `s` contains the substring `sub`.

```
print(contains("hello", "ell"))    # true
print("hello".contains("xyz"))     # false (UFCS)
```

---

starts_with

Signature: `starts_with(s: str, prefix: str) -> bool`

Returns whether string `s` starts with `prefix`.

```
print(starts_with("hello", "hel"))    # true
print("hello".starts_with("world"))  # false (UFCS)
```

---

ends_with

Signature: `ends_with(s: str, suffix: str) -> bool`

Returns whether string `s` ends with `suffix`.

```
print(ends_with("hello", "llo"))    # true
print("hello".ends_with("world"))  # false (UFCS)
```

---

find

Signature: `find(s: str, sub: str) -> Option<int>`

Returns the character position of the first occurrence of substring `sub` in string `s`. Returns `None` if not found.

```
print(find("hello world", "world"))    # Some(6)
print(find("hello", "xyz"))            # None
print("abcdef".find("cd"))             # Some(2) (UFCS)
```

---

substring

Signature: `substring(s: str, start: int, end: int) -> str`

Returns the substring of `s` from `start` to `end` (exclusive). Indices are character positions (UTF-8 aware).

```
print(substring("hello world", 0, 5))   # hello
print(substring("hello world", 6, 11))  # world
print("abcdef".substring(1, 4))         # bcd (UFCS)
```

---

char_at

Signature: `char_at(s: str, i: int) -> str`

Returns the UTF-8 character at position `i` in string `s` as a string.

```
print(char_at("hello", 0))   # h
print("abc".char_at(2))       # c (UFCS)
```

---

replace

Signature: `replace(s: str, old: str, new: str) -> str`

Returns a new string with all occurrences of `old` in `s` replaced with `new`.

```
print(replace("hello world", "world", "ry"))   # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))      # baz bar baz (UFCS)
```

---

to_upper

Signature: `to_upper(s: str) -> str`

Returns a new string with ASCII lowercase letters (a-z) converted to uppercase.

```
print(to_upper("hello"))         # HELLO
print("Hello World".to_upper())  # HELLO WORLD (UFCS)
```

---

to_lower

Signature: `to_lower(s: str) -> str`

Returns a new string with ASCII uppercase letters (A-Z) converted to lowercase.

```
print(to_lower("HELLO"))         # hello
print("Hello World".to_lower())  # hello world (UFCS)
```

---

trim

Signature: `trim(s: str) -> str`

Returns a new string with leading and trailing whitespace characters (spaces, tabs, newlines, carriage returns) removed.

```
print(trim("  hello  "))   # hello
print("  hi  ".trim())      # hi (UFCS)
```

---

trim_start

Signature: `trim_start(s: str) -> str`

Returns a new string with leading whitespace characters removed.

```
print(trim_start("  hello  "))   # hello
print("  hi".trim_start())       # hi (UFCS)
```

---

trim_end

Signature: `trim_end(s: str) -> str`

Returns a new string with trailing whitespace characters removed.

```
print(trim_end("  hello  "))   #   hello
print("hi  ".trim_end())       # hi (UFCS)
```

---

repeat

Signature: `repeat(s: str, n: int) -> str`

Returns a new string with `s` repeated `n` times.

```
print(repeat("ab", 3))     # ababab
print("ha".repeat(3))      # hahaha (UFCS)
```

---

reverse

Signature: `reverse(s: str) -> str`

Returns a new string with the characters reversed (UTF-8 aware).

```
print(reverse("hello"))    # olleh
print("abc".reverse())     # cba (UFCS)
```

---

byte_len

Signature: `byte_len(s: str) -> int`

Returns the byte length of string `s`. Unlike `len()`, which returns the number of UTF-8 characters, `byte_len()` returns the number of bytes.

```
print(byte_len("hello"))   # 5
print(byte_len("  "))   # 9
print(len("  "))        # 3 (characters)
```

---

split

Signature: `split(s: str, delim: str) -> List<str>`

Splits string `s` by delimiter `delim` and returns a `List<str>`.

```
let parts = split("a,b,c", ",")
print(parts[0])   # a
print(parts[1])   # b
print(parts[2])   # c

for word in "hello world".split(" "):
    print(word)
# hello
# world
```

---

join

Signature: `join(xs: List<str>, sep: str) -> str`

Joins the elements of a string list with separator `sep` and returns a string.

```
let parts = ["a", "b", "c"]
print(join(parts, ","))         # a,b,c
print(parts.join("-"))          # a-b-c (UFCS)
```

---

to_int

Signature: `to_int(s: str) -> int`

Converts a string to an integer.

```
print(to_int("42"))       # 42
print(to_int("-7"))       # -7
print("123".to_int())     # 123 (UFCS)
```

---

to_float

Signature: `to_float(s: str) -> float`

Converts a string to a floating-point number.

```
print(to_float("3.14"))   # 3.14
print("2.5".to_float())   # 2.5 (UFCS)
```

---

to_str

Signature: `to_str(v: int | float | bool | str) -> str`

Converts a value to a string.

Type	Conversion Format
`int`	`%ld`
`float`	`%g`
`bool`	`"true"` / `"false"`
`str`	Returned as-is

```
print(to_str(42))          # 42
print(to_str(3.14))        # 3.14
print(to_str(true))        # true
print(99.to_str())         # 99 (UFCS)
```

English | |

Built-in Function Reference

Function List

Core

Function	Description
print(expr)	Prints a value to standard output
len(x)	Returns the number of elements in a list, map, or set, or the number of UTF-8 characters in a string
range(n) / range(start, end) / range(start, end, step)	Generates a list of integers
exit(code)	Terminates the process with the given exit code
args()	Returns command-line arguments as List<str>
available_parallelism()	Returns the runtime worker count as int
channel[T]()	Creates an unbuffered Channel<T>
channel[T](capacity)	Creates a buffered Channel<T>
send(ch, value)	Sends a value through Channel<T>
try_send(ch, value)	Attempts to send through Channel<T> without blocking
recv(ch)	Receives a value from Channel<T>
recv_opt(ch)	Receives from Channel<T> as Option<T> or bool for Channel<Unit>
try_recv(ch)	Attempts to receive from Channel<T> as Option<T> or bool for Channel<Unit>
close(ch)	Closes a Channel<T>
join(task)	Waits for a Task<T> to complete and returns its result

Option

Function	Description
Some(expr)	Constructs the value-present variant of an Option type

Collection Operations

Function	Description
has_key(map, key)	Returns whether a key exists in the map
add(set, value)	Adds an element to a set (duplicates are ignored)
remove(set, value)	Removes an element from a set
append(list, value) / append!(list, value)	Adds an element to the end of a list (mutating)
appended(list, value)	Returns a new list with the element added (non-mutating)

Function	Description
pop(list)	Removes and returns the last element as Option<T>
reverse(list)	Returns a new reversed list (also works on strings)
reverse!(list)	Reverses a list in place (mutating)
slice(list, start, end)	Returns a new sub-list from start to end
take(list, n)	Returns a new list with the first n elements
tap(list, fn)	Calls fn on each element for side effects, returns the original list
filter(list, pred)	Returns a new list with elements matching the predicate
map(list, fn)	Returns a new list with each element transformed
sort(list) / sort(list, comp)	Returns a new sorted list (default ascending)
sort!(list) / sort!(list, comp)	Sorts a list in place (mutating)
insert(list, i, val)	Inserts an element at index i
remove_at(list, i)	Removes and returns the element at index i
items(map)	Returns a list of (key, value) tuples
remove(map, key)	Removes the entry with the specified key
get(map, key)	Returns the value for key as Option<V>
get(map, key, default)	Returns the value for key, or default if not found
union(set, set)	Returns the union of two sets
intersection(set, set)	Returns the intersection of two sets
difference(set, set)	Returns the difference of two sets
symmetric_difference(set, set)	Returns the symmetric difference of two sets
is_subset(set, set)	Returns whether the first set is a subset of the second
is_superset(set, set)	Returns whether the first set is a superset of the second

Iterator

Function	Description
iter(collection)	Creates a lazy iterator from a List, Set, or Map
next(iter)	Returns the next element as Option<T>, or None if exhausted
to_list(iter)	Collects all remaining iterator elements into a List<T>
filter(iter, pred)	Returns a lazy iterator that yields only elements matching the predicate
map(iter, fn)	Returns a lazy iterator that transforms each element
take(iter, n)	Returns a lazy iterator that yields at most n elements

String Operations

Function	Description
contains(s, sub)	Whether a substring is contained
starts_with(s, prefix)	Whether it starts with a prefix
ends_with(s, suffix)	Whether it ends with a suffix
find(s, sub)	Character position of a substring (Option<int>)
byte_len(s)	Returns the byte length of a string
substring(s, start, end)	Extract a substring
char_at(s, i)	Get the character at a specified position

Function	Description
`replace(s, old, new)`	Replace all occurrences of a substring
`to_upper(s)` / `to_lower(s)`	Uppercase / lowercase conversion
`trim(s)` / `trim_start(s)` / `trim_end(s)`	Whitespace removal
`repeat(s, n)`	Repeat a string n times
`reverse(s)`	Reverse a string
`split(s, delim)`	Split a string into a list
`join(list, sep)`	Join list elements with a separator
`to_int(s)` / `to_float(s)` / `to_str(v)`	Type conversion

-> See **String Operation Function Reference** for details

---

print

Signature: `print(expr)`

Prints a value to standard output. A newline is appended at the end.

Type	Output Format
`int`	`%ld`
`float`	`%g`
`bool`	`true` / `false`
`str`	`%s`
`Option` (Some)	`Some(value)`
`Option` (None)	`None`
`list`	`[elem1, elem2, ...]`
`map`	`{key1: val1, key2: val2, ...}`
`set`	`{elem1, elem2, ...}`
`enum`	Variant name (e.g., `Red`)

```
print(42)           # 42
print(3.14)         # 3.14
print(true)         # true
print("hello")      # hello
print(Some(1))      # Some(1)
print(None)         # None
print([1, 2, 3])    # [1, 2, 3]
print({"a": 1})     # {a: 1}
print({1, 2, 3})    # {1, 2, 3}
```

Error condition: Passing a struct or tuple directly causes a compile error.

Some

Signature: `Some(expr) -> Option<T>`

Constructs the value-present variant of an Option type.

```
let x: Option<int> = Some(42)
print(x)    # Some(42)
```

---

len

Signature: `len(x: List<T> | Map<K, V> | Set<T> | str) -> int`

Returns the number of elements in a list, map, or set, or the number of UTF-8 characters in a string. Use `byte_len()` for the byte length.

```
print(len([1, 2, 3]))          # 3
print(len({"a": 1, "b": 2})) # 2
print(len({1, 2, 3}))          # 3
print(len("hello"))            # 5
print(len("  "))            # 3 (UTF-8 characters)
```

---

has_key

Signature: `has_key(m: Map<K, V>, key: K) -> bool`

Returns whether a specified key exists in the map. UFCS notation is also available.

```
let m = {"a": 1, "b": 2}
print(has_key(m, "a"))     # true
print(m.has_key("z"))      # false (UFCS)
```

---

add

Signature: `add(s: Set<T>, value: T)`

Adds an element to a set. Does nothing if the element already exists. UFCS notation is also available.

```
let s = {1, 2, 3}
s.add(4)           # UFCS
add(s, 5)          # Normal call
s.add(1)           # Ignored because it already exists
print(len(s))      # 5
```

---

remove

Signature: `remove(s: Set<T>, value: T)`

Removes an element from a set. UFCS notation is also available.

```
let s = {1, 2, 3}
s.remove(2)        # UFCS
print(2 in s)      # false
```

---

range

Signature: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

Generates a list of integers.

Form	Generated Values
`range(n)`	`[0, 1, ..., n-1]`
`range(start, end)`	`[start, start+1, ..., end-1]`
`range(start, end, step)`	`[start, start+step, start+2*step, ...]` (up to but not including `end`)

- When `step > 0`, generates values from `start` ascending toward `end`.
- When `step < 0`, generates values from `start` descending toward `end`.
- When `step == 0`, a runtime error occurs.
- If the range is empty (e.g., `range(0, 10, -1)`), returns an empty list.

```
print(range(3))           # [0, 1, 2]
print(range(2, 5))        # [2, 3, 4]
print(range(0, 10, 2))    # [0, 2, 4, 6, 8]
print(range(10, 0, -3))   # [10, 7, 4, 1]

for i in range(3):
    print(i)
# 0
# 1
# 2
```

---

exit

Signature: `exit(code: int)`

Terminates the process immediately with the given exit code. Code after `exit()` is unreachable.

```
exit(0)        # normal termination
exit(1)        # error termination
```

---

args

Signature: `args() -> List<str>`

Returns the command-line arguments passed to the script as a list of strings. Does not include the interpreter name or the script filename — only the arguments after the script path.

```
# Run: ry script.ry hello world
let a = args()
print(len(a))     # 2
print(a[0])       # hello
print(a[1])       # world

for x in args():
    print(x)
```

---

append

Signature: `append(list: List<T>, value: T)`

Adds an element to the end of a list. This is a mutating operation — the list is modified in place. UFCS notation is also available.

```
var xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

---

pop

Signature: `pop(list: List<T>) -> Option<T>`

Removes and returns the last element of a list as `Option<T>`. Returns `None` if the list is empty. UFCS notation is also available.

```
var xs = [1, 2, 3]
let v = xs.pop()
print(v)     # Some(3)
print(xs)    # [1, 2]
```

---

reverse (list)

Signature: `reverse(list: List<T>) -> List<T>`

Returns a new list with elements in reverse order. The original list is not modified. Also works on strings (see String Operations). UFCS notation is also available.

```
let xs = [1, 2, 3]
let ys = reverse(xs)
print(ys)    # [3, 2, 1]
print(xs)    # [1, 2, 3] (unchanged)
```

---

slice

Signature: `slice(list: List<T>, start: int, end: int) -> List<T>`

Returns a new sub-list from `start` (inclusive) to `end` (exclusive). Indices are clamped to the valid range (`0` to `len(list)`). UFCS notation is also available.

```
let xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))      # [2, 3]
print(slice(xs, 0, 100))    # [1, 2, 3, 4, 5] (clamped)
```

---

take

Signature: `take(list: List<T>, n: int) -> List<T>`

Returns a new list with the first `n` elements. If `n` exceeds the list length, returns a copy of the entire list. If `n <= 0`, returns an empty list. The original list is not modified. UFCS notation is also available.

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.take(3)
print(ys)    # [1, 2, 3]
```

```
print(xs.take(10))    # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))     # []
```

---

tap

Signature: `tap(list: List<T>, fn: fn(T) -> R) -> List<T>`

Calls the given function on each element (ignoring any return value), then returns the original list unchanged. Useful for debugging or inserting side effects in a method chain. UFCS notation is also available.

```
let xs = [1, 2, 3]
let ys = xs.tap(fn(x: int): print(x)).map(fn(x: int): x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

---

filter

Signature: `filter(list: List<T>, pred: fn(T) -> bool) -> List<T>`

Returns a new list containing only elements for which the predicate returns `true`. The original list is not modified. UFCS notation is also available.

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.filter((x: int) -> x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5]  (unchanged)
```

---

map

Signature: `map(list: List<T>, fn: fn(T) -> U) -> List<U>`

Returns a new list with each element transformed by the given function. The output element type can differ from the input type. The original list is not modified. UFCS notation is also available.

```
let xs = [1, 2, 3]
let ys = xs.map((x: int) -> x * 2)
print(ys)    # [2, 4, 6]
```

---

sort

Signature: `sort(list: List<T>) -> List<T>` / `sort(list: List<T>, comp: fn(T, T) -> bool) -> List<T>`

Returns a new sorted list. Default is ascending order. An optional comparator function can be provided that returns `true` if the first argument should come before the second. The original list is not modified. The sort is **stable** (equal elements preserve their original order). UFCS notation is also available.

```
let xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]

# Descending order
let desc = xs.sort((a: int, b: int) -> a > b)
print(desc)    # [3, 2, 1]
```

---

sort!

Signature: `sort!(list: List<T>)` / `sort!(list: List<T>, comp: fn(T, T) -> bool)`

Sorts a list in place. Same sorting algorithm as `sort()`, but modifies the original list instead of creating a new one. UFCS notation is also available.

```
var xs = [3, 1, 2]
xs.sort!()
print(xs)   # [1, 2, 3]
```

---

reverse!

Signature: `reverse!(list: List<T>)`

Reverses a list in place. UFCS notation is also available.

```
var xs = [1, 2, 3]
xs.reverse!()
print(xs)   # [3, 2, 1]
```

---

appended

Signature: `appended(list: List<T>, value: T) -> List<T>`

Returns a new list with the element added at the end. The original list is not modified. UFCS notation is also available.

```
let xs = [1, 2]
let ys = xs.appended(3)
print(xs)   # [1, 2] (unchanged)
print(ys)   # [1, 2, 3]
```

---

append!

Signature: `append!(list: List<T>, value: T)`

Alias for `append()`. Adds an element to the end of a list in place. Provided for naming consistency with the `!` convention.

---

first

Signature: `first(list: List<T>) -> Option<T>`

Returns the first element of a list as `Option<T>`. Returns `None` if the list is empty.

```
print(first([10, 20, 30]))   # Some(10)
```

---

last

Signature: last(list: List<T>) -> Option<T>

Returns the last element of a list as Option<T>. Returns None if the list is empty.

```
print(last([10, 20, 30]))    # Some(30)
```

---

get (Map)

Signature: get(map: Map<K, V>, key: K) -> Option<V> / get(map: Map<K, V>, key: K, default: V) -> V

Two-argument form returns the value for key as Option<V>. Three-argument form returns the value or the default.

```
let m = {"a": 1, "b": 2}
print(get(m, "a"))       # Some(1)
print(get(m, "z"))       # None
print(get(m, "z", 0))    # 0
```

---

iter

Signature: iter(collection: List<T> | Set<T>) -> Iterator<T> / iter(collection: Map<K, V>) -> Iterator<(K, V)>

Creates a lazy iterator from a collection. The iterator does not copy data; it references the original collection. UFCS notation is also available.

- For List<T> and Set<T>, the element type is T.
- For Map<K, V>, the element type is the tuple (K, V).

```
let xs = [1, 2, 3]
let it = xs.iter()          # Iterator<int>
let ys = it.to_list()       # [1, 2, 3]

let m = {"a": 1, "b": 2}
for k, v in m.iter():       # Iterator<(str, int)>
    print(k)
```

---

next

Signature: next(iter: Iterator<T>) -> Option<T>

Returns the next element from the iterator as Option<T>. Returns None when the iterator is exhausted. The iterator advances its internal state on each call. UFCS notation is also available.

```
let it = [10, 20].iter()
print(it.next())    # Some(10)
print(it.next())    # Some(20)
print(it.next())    # None
```

---

to_list

Signature: `to_list(iter: Iterator<T>) -> List<T>`

Collects all remaining elements from the iterator into a new list. UFCS notation is also available.

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.iter().filter(fn(x: int): x > 2).to_list()
print(ys)   # [3, 4, 5]
```

English | |

Collection Reference (Tuple, List, Map, Set)

Tuple

Overview

A fixed-length combination of heterogeneous values. Implemented as a stack-allocated value type using LLVM literal StructType.

Syntax

```
let t = (1, 3.14)
let t: (int, float) = (1, 3.14)
```

Type Annotation

```
let pair: (int, str) = (42, "hello")
let triple: (int, float, bool) = (1, 2.0, true)
```

Element Access

Access elements using numeric indices `.0`, `.1`, etc.

```
let t = (10, 3.14)
print(t.0)   # 10
print(t.1)   # 3.14
```

Function Return Values

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)

let result = swap(1, 2)
print(result.0)   # 2
print(result.1)   # 1
```

Constraints and Errors

Constraint	Details
Out-of-range index	Compile error
Passing a tuple directly to `print`	Compile error (not supported by print)

List

Overview

A variable-length sequence of elements of the same type. Allocated on the heap.

Syntax

```
let xs = [1, 2, 3]
let xs: List<int> = [1, 2, 3]
```

Supported Element Types

`int`, `float`, `bool`, `str`

Index Access

```
let xs = [1, 2, 3]
print(xs[0])   # 1
print(xs[2])   # 3
```

Index Assignment

```
let xs = [1, 2, 3]
xs[0] = 99
print(xs[0])   # 99
```

len

```
let xs = [1, 2, 3]
print(len(xs))   # 3
```

print

```
let xs = [1, 2, 3]
print(xs)   # [1, 2, 3]
```

for Iteration

```
let xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

append

Adds an element to the end of the list. This is a mutating operation.

```
var xs = [1, 2]
xs.append(3)
print(xs)   # [1, 2, 3]
```

pop

Removes and returns the last element. Causes a runtime error on an empty list.

```
var xs = [1, 2, 3]
let v = xs.pop()
print(v)    # 3
print(xs)   # [1, 2]
```

reverse

Returns a new list with elements in reverse order. The original list is not modified. Also works on strings.

```
let xs = [1, 2, 3]
print(reverse(xs))   # [3, 2, 1]
print(xs)            # [1, 2, 3] (unchanged)
```

slice

Returns a new sub-list from `start` (inclusive) to `end` (exclusive). Indices are clamped to the valid range.

```
let xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))     # [2, 3]
print(slice(xs, 0, 100))   # [1, 2, 3, 4, 5] (clamped)
```

take

Returns a new list with the first `n` elements. If `n` exceeds the list length, returns a copy of the entire list. If `n <= 0`, returns an empty list. The original list is not modified.

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.take(3)
print(ys)   # [1, 2, 3]
print(xs.take(10))   # [1, 2, 3, 4, 5] (clamped)
print(xs.take(0))    # []
```

tap

Calls the given function on each element (ignoring any return value), then returns the original list unchanged. Useful for debugging or inserting side effects in a method chain.

```
let xs = [1, 2, 3]
let ys = xs.tap(fn(x: int): print(x)).map(fn(x: int): x * 2)
# prints 1, 2, 3, then ys = [2, 4, 6]
```

filter

Returns a new list containing only elements that satisfy the predicate. The original list is not modified.

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.filter(fn(x: int): x > 3)
print(ys)   # [4, 5]
```

map

Returns a new list with each element transformed by the given function. The output element type can differ from the input. The original list is not modified.

```
let xs = [1, 2, 3]
let ys = xs.map(fn(x: int): x * 2)
print(ys)   # [2, 4, 6]
```

sort

Returns a new sorted list. Default is ascending order. A custom comparator can be provided. The original list is not modified. The sort is **stable** (equal elements preserve their original order). Internally uses TimSort.

```
let xs = [3, 1, 2]
print(xs.sort())   # [1, 2, 3]

# Descending order with comparator
let desc = xs.sort(fn(a: int, b: int): a > b)
print(desc)   # [3, 2, 1]
```

Chaining filter, map, sort

These functions return new lists, so they can be chained via UFCS.

```
let xs = [5, 3, 1, 4, 2]
let result = xs.filter(fn(x: int): x > 1).map(fn(x: int): x * 10).sort()
print(result)   # [20, 30, 40, 50]
```

reduce

Reduces a list to a single value using an accumulator function, starting with the first element.

```
let xs = [1, 2, 3, 4, 5]
let total = reduce(xs, fn(a: int, b: int): a + b)
print(total)   # 15
```

fold

Folds a list to a single value using an accumulator function and an explicit initial value.

```
let xs = [1, 2, 3, 4, 5]
let total = fold(xs, 0, fn(a: int, b: int): a + b)
print(total)   # 15
```

any

Returns `true` if at least one element satisfies the predicate.

```
let xs = [1, 2, 3, 4, 5]
print(any(xs, fn(x: int): x > 4))   # true
print(any(xs, fn(x: int): x > 9))   # false
```

all

Returns `true` if every element satisfies the predicate.

```
let xs = [2, 4, 6]
print(all(xs, fn(x: int): x > 0))   # true
print(all(xs, fn(x: int): x > 3))   # false
```

sum

Returns the sum of all elements.

```
let xs = [1, 2, 3, 4, 5]
print(sum(xs))   # 15
```

min

Returns the minimum element.

```
let xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

max

Returns the maximum element.

```
let xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

first

Returns the first element. Causes a runtime error on an empty list.

```
let xs = [10, 20, 30]
print(first(xs))    # 10
```

last

Returns the last element. Causes a runtime error on an empty list.

```
let xs = [10, 20, 30]
print(last(xs))    # 30
```

is_empty

Returns `true` if the list has no elements.

```
let xs = [1, 2, 3]
print(is_empty(xs))    # false
print(is_empty([]))    # true (requires type annotation in practice)
```

enumerate

Returns a list of `(index, element)` tuples.

```
let xs = [10, 20, 30]
let pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# Tuple destructuring in for loop
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

zip

Combines two lists into a list of `(elem1, elem2)` tuples. The result length equals the shorter list.

```
let xs = [1, 2, 3]
let ys = ["a", "b", "c"]
let pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# Tuple destructuring in for loop
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

insert

Inserts an element at the specified index. Elements at and after the index are shifted right.

```
var xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)   # [1, 99, 2, 3]
```

remove_at

Removes and returns the element at the specified index. Elements after the index are shifted left.

```
var xs = [1, 2, 3, 4]
let v = remove_at(xs, 1)
print(v)    # 2
print(xs)   # [1, 3, 4]
```

remove

Removes the first occurrence of the specified value from the list. Does nothing if the value is not found. This is a mutating operation.

```
var xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)   # [1, 3, 2, 4]
```

distinct

Returns a new list with duplicate elements removed. The original order is preserved (first occurrence kept). The original list is not modified.

```
let xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))   # [1, 2, 3, 4]
print(xs)             # [1, 2, 3, 2, 1, 4] (unchanged)
```

flatten

Flattens a nested list (list of lists) by one level. Returns a new list. The original list is not modified.

```
let xs = [[1, 2], [3, 4]]
print(flatten(xs))   # [1, 2, 3, 4]
print(xs)            # [[1, 2], [3, 4]] (unchanged)
```

Operation Complexity

Operation	Complexity
`xs[i]` index access	O(1)
`append` / `append!`	O(1) amortized
`pop`	O(1)
`first`, `last`	O(1)
`insert`, `remove_at`	O(n)
`sort` / `sort!`	O(n log n)
`take`	O(n)
`tap`	O(n)
`filter`, `map`, `reduce`, `fold`	O(n)
`reverse` / `reverse!`	O(n)
`distinct`	O(n)

Operation	Complexity
`len`	O(1)

Constraints and Errors

Constraint	Details
All elements must be the same type	Mixed types cause a compile error
Empty list `[]`	Compile error because type cannot be inferred
Out-of-range access	Runtime error (exit(1))

Map

Overview

A key-value mapping. Allocated on the heap.

Syntax

```
let m = {"a": 1, "b": 2}
let m: Map<str, int> = {"a": 1, "b": 2}
```

Key Access

```
let m = {"a": 1, "b": 2}
print(m["a"])   # 1
```

Insert and Update

```
let m = {"a": 1}
m["b"] = 2      # Insert new entry
m["a"] = 99     # Update existing entry
```

len

```
let m = {"a": 1, "b": 2, "c": 3}
print(len(m))   # 3
```

print

```
let m = {"a": 1, "b": 2}
print(m)   # {a: 1, b: 2}
```

has_key

```
let m = {"a": 1, "b": 2}
print(m.has_key("a"))   # true
print(m.has_key("z"))   # false
```

keys

Returns a list of all keys in the map.

```
let m = {"a": 1, "b": 2, "c": 3}
print(keys(m))   # ["a", "b", "c"]
```

values

Returns a list of all values in the map.

```
let m = {"a": 1, "b": 2, "c": 3}
print(values(m))    # [1, 2, 3]
```

items

Returns a list of (key, value) tuples for all entries in the map.

```
let m = {"a": 1, "b": 2}
let pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

remove (Map)

Removes the entry with the specified key from the map. Does nothing if the key does not exist.

```
let m = {"a": 1, "b": 2}
remove(m, "a")
print(m)    # {b: 2}
```

get

Returns the value for the specified key, or a default value if the key does not exist.

```
let m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

merge

Returns a new map that combines all entries from both maps. When keys overlap, values from the second map take precedence. The original maps are not modified.

```
let m1 = {"a": 1, "b": 2}
let m2 = {"b": 99, "c": 3}
let m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3
```

Constraints and Errors

Constraint	Details
All keys must be the same type	Mixed key types cause a compile error
All values must be the same type	Mixed value types cause a compile error
Empty map	Type annotation is required (e.g., `let m: Map<str, int> = {"a": 1}`)
Accessing a non-existent key	Runtime error (exit(1))
Key lookup	Hash table (O(1) average)
Capacity overflow	Automatically doubles in size

Set

Overview

A collection that holds elements of the same type without duplicates. Allocated on the heap.

Syntax

```
let s = {1, 2, 3}
let s: Set<int> = {1, 2, 3}
```

Supported Element Types

`int`, `float`, `bool`, `str`

in Operator (Membership Check)

```
let s = {1, 2, 3}
print(2 in s)   # true
print(5 in s)   # false
```

len

```
let s = {1, 2, 3}
print(len(s))   # 3
```

print

```
let s = {1, 2, 3}
print(s)   # {1, 2, 3}
```

add (Add Element)

Duplicate elements are ignored when added.

```
let s = {1, 2, 3}
s.add(4)         # Add
s.add(1)         # Ignored because it already exists
print(len(s))    # 4
```

remove (Remove Element)

```
let s = {1, 2, 3}
s.remove(2)
print(2 in s)   # false
```

for Iteration

```
let s = {10, 20, 30}
for x in s:
    print(x)
```

Empty Set

An empty set requires a type annotation.

```
let s: Set<int> = {}
```

Function Parameters

```
fn has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

union

Returns a new set containing all elements from both sets.

```
let a = {1, 2, 3}
let b = {3, 4, 5}
print(union(a, b))   # {1, 2, 3, 4, 5}
```

intersection

Returns a new set containing only elements present in both sets.

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(intersection(a, b))   # {2, 3}
```

difference

Returns a new set containing elements in the first set but not in the second.

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(difference(a, b))   # {1}
```

symmetric_difference

Returns a new set containing elements that are in either set but not in both.

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(symmetric_difference(a, b))   # {1, 4}
```

is_subset

Returns `true` if all elements of the first set are contained in the second set.

```
let a = {1, 2}
let b = {1, 2, 3}
print(is_subset(a, b))   # true
print(is_subset(b, a))   # false
```

is_superset

Returns `true` if the first set contains all elements of the second set.

```
let a = {1, 2, 3}
let b = {1, 2}
print(is_superset(a, b))   # true
print(is_superset(b, a))   # false
```

Constraints and Errors

Constraint	Details
All elements must be the same type	Mixed types cause a compile error
Empty set `{}`	Type annotation is required
Element lookup	Hash table (O(1) average)
Capacity overflow	Automatically doubles in size

Iterator

Overview

A lazy iterator abstraction that enables efficient data transformation pipelines. Iterators do not copy or materialize intermediate results — each element is processed on demand.

Creating Iterators

Use `iter()` to create an iterator from any collection:

```
let xs = [1, 2, 3, 4, 5]
let it = xs.iter()           # Iterator<int>

let s = {10, 20, 30}
let sit = s.iter()           # Iterator<int>

let m = {"a": 1, "b": 2}
let mit = m.iter()           # Iterator<(str, int)>
```

Lazy Method Chaining

Iterator methods return new iterators, forming a pipeline that is only evaluated when consumed:

Method	Description
`.filter(fn)`	Yields only elements where the predicate returns `true`
`.map(fn)`	Transforms each element using the given function
`.take(n)`	Yields at most `n` elements

```
let result = [1, 2, 3, 4, 5]
    .iter()
    .filter(fn(x: int): x > 2)
    .map(fn(x: int): x * 2)
    .take(2)
    .to_list()   # [6, 8]
```

Consuming Iterators

Method	Description
`.to_list()`	Collects all elements into a `List<T>`
`.next()`	Returns the next element as `Option<T>`

```
let it = [10, 20].iter()
print(it.next())   # Some(10)
print(it.next())   # Some(20)
print(it.next())   # None
```

For Loop Support

Iterators can be used directly in `for` loops:

```
for x in [1, 2, 3].iter().filter(fn(x: int): x > 1):
    print(x)
# 2
# 3

for k, v in {"a": 1, "b": 2}.iter():
    print(k)
```

English | |

Design by Contract (DbC)

Ry supports Eiffel-style Design by Contract with preconditions (`require`), postconditions (`ensure`), and struct invariants (`invariant`). Contract violations terminate the process with `exit(1)`.

Preconditions (`require`)

Preconditions are checked at function entry. They specify what must be true for the function to be called correctly.

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    var new_balance: int = balance + amount
    return new_balance
```

If any precondition fails, the program terminates with:

```
Contract violation: require failed in deposit()
```

Postconditions (`ensure`)

Postconditions are checked before every `return`. They specify what the function guarantees about its result.

`result` keyword

Inside an `ensure` block, `result` refers to the return value.

```
fn abs(x: int) -> int:
    ensure:
        result >= 0
    if x < 0:
        return -x
    return x
```

`old()` expression

`old(expr)` captures the value of an expression at function entry, before the function body executes. This is useful for comparing pre- and post-states.

```
fn increment(x: int) -> int:
    ensure:
        result == old(x) + 1
    return x + 1
```

Combined Example

```
fn deposit(amount: int, balance: int) -> int:
    require:
        amount > 0
        balance >= 0
    ensure:
        result >= 0
        result == old(balance) + amount
    var new_balance: int = balance + amount
    return new_balance
```

Struct Invariants (`invariant`)

Invariants are conditions that must always hold for a struct instance. They are checked: - After construction - After every field assignment

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance
```

```
let a = BankAccount(100, 0)     # OK: 100 >= 0
a.balance = -1                   # Contract violation: invariant failed
```

Rules

- `require` and `ensure` blocks are optional and appear before the function body.
- `require` must come before `ensure` when both are present.
- `result` and `old()` can only be used inside `ensure` blocks.
- `invariant` appears at the end of a `record` definition, after all field declarations.
- All contract violations terminate with `exit(1)` and print a diagnostic message.

English | |

Control Flow Reference

if / elif / else

Syntax

```
if condition:
    # then block
elif condition:
    # elif block (can have multiple)
```

```
else:
    # else block (optional)
```

Condition Types

Type	Falsy Value	Truthy Value
`bool`	`false`	`true`
`int`	`0`	non-zero

`float` and `str` cannot be used directly as conditions.

Example

```
let x = 10

if x > 5:
    print("big")
elif x == 5:
    print("five")
else:
    print("small")
```

Scope Rules

- Each `if` / `elif` / `else` block has its own independent block scope.
- Variables declared inside a block are not accessible outside the block.

```
if true:
    let y = 42
# y is not accessible here
```

---

while

Syntax

```
while condition:
    # loop body
```

Repeats the loop body while the condition is `true`.

Example

```
let i = 0
while i < 5:
    print(i)
    i += 1
```

Combining with break / continue

```
let i = 0
while true:
    if i >= 3:
        break
    i += 1
```

for

Syntax

```
# List / set iteration
for x in iterable_expr:
    # x is assigned each element

# range (starting from 0)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (with start and end)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (with step)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...
```

Map Key-Value Iteration

```
for k, v in map_expr:
    # k is the key, v is the value for each entry
```

Tuple Destructuring

When iterating over a list of 2-element tuples (e.g. from `enumerate()` or `zip()`), you can destructure into two variables. Use `_` to discard a value.

```
let xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)          # 11, 22

for _, x in enumerate(xs):
    print(x)              # index discarded
```

Range Operator (`..`)

The `..` operator creates an inclusive integer range. `1 .. 5` produces `[1, 2, 3, 4, 5]`.

```
for i in 1 .. 5:
    print(i)     # 1 2 3 4 5
```

Example

```
let xs = [10, 20, 30]
for x in xs:
    print(x)

let s = {1, 2, 3}
```

```
for x in s:
    print(x)

for i in range(5):
    print(i)     # 0 1 2 3 4

for i in range(2, 6):
    print(i)     # 2 3 4 5

for i in range(0, 10, 2):
    print(i)     # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)     # 10 7 4 1

# Map iteration
let m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

# Range operator
for i in 1 .. 3:
    print(i)     # 1 2 3
```

---

spawn / await

`spawn` starts a user-defined function or lambda call on the runtime worker pool and returns `Task<T>`. `await` and `join(task)` both wait for a task to complete.

```
fn square(x: int) -> int:
    return x * x

let t: Task<int> = spawn square(12)
print(await t)          # 144
let u: Task<int> = spawn square(3)
print(join(u))          # 9, function-form equivalent of await
```

Constraints

- `spawn` only accepts a function-call expression.
- The callee must be a user-defined function or lambda.
- `spawn` does not support `Unit`-returning calls in v1.
- Spawned work runs as lightweight runtime jobs rather than one OS thread per task.
- `await` requires a `Task<T>` value.
- `await expr` can be used as either an expression or a statement.

channels

`Channel<T>` is the built-in blocking communication primitive for passing values between tasks.

```
fn worker(ch: Channel<int>) -> int:
    send(ch, 42)
    close(ch)
```

```
    return 0

let ch: Channel<int> = channel[int]()
let t: Task<int> = spawn worker(ch)
for x in ch:
    print(x)
print(join(t))
```

Rules

- `channel[T]()` creates an unbuffered channel.
- `channel[T](n)` creates a buffered channel with capacity `n`.
- `send(ch, value)` blocks until the value is accepted.
- `try_send(ch, value)` attempts an immediate send and returns `bool`.
- `recv(ch)` blocks until a value is available and remains the strict receive API.
- `recv_opt(ch)` blocks until a value is available or the channel is closed and drained.
- `try_recv(ch)` attempts an immediate receive and returns `Option<T>` or `bool` for `Channel<Unit>`.
- `for x in ch:` iterates values until the channel is closed and drained.
- `close(ch)` closes the channel.
- In v1, `send` on a closed channel and `recv` on an empty closed channel raise a runtime error.
- `recv_opt(ch: Channel<T>) -> Option<T>` returns `Some(v)` for received values and `None` once the channel is closed and drained.
- `recv_opt(ch: Channel<Unit>) -> bool` returns `true` for a received `Unit` value and `false` once the channel is closed and drained.
- `try_recv(ch: Channel<T>) -> Option<T>` returns `Some(v)` if a value is immediately available and `None` otherwise, including closed-and-drained channels.
- `try_recv(ch: Channel<Unit>) -> bool` returns `true` if a `Unit` value is immediately available and `false` otherwise.
- `for _ in ch:` can be used to consume `Channel<Unit>`.

select

`select` waits on multiple channel operations and executes exactly one ready branch.

```
select:
    case let x = recv(inbox):
        print(x)
    case send(outbox, 42):
        print("sent")
    else:
        print("idle")
```

Rules

- `select` is a statement, not an expression.
- Cases must be `case let name = recv(ch):`, `case let name = recv_opt(ch):`, or `case send(ch, value):`.
- Use `let _ = recv(ch)` to discard a received value.
- `recv_opt` cases bind `Option<T>` or `bool` for `Channel<Unit>`.
- `else:` is optional, and if present it must be the last branch.
- `timeout n:` is an optional last branch that waits up to `n` milliseconds for the whole `select`.
- `else` and `timeout` cannot be used together.
- Without `else`, `select` blocks until one case becomes ready.
- In v1, closed-channel errors are preserved inside `select`: `send` on a closed channel and `recv` on an empty closed channel still raise runtime errors.

`@parallel for`

`@parallel` can be attached only to counted `for` loops over `range(...)` or integer `..` ranges. The loop body runs in parallel chunks on the runtime worker pool.

```
@parallel
for i in range(8):
    print(i)
```

Constraints

- Only `range(...)` and integer `..` loops are supported.
- Destructuring iteration is not supported.
- Assigning to outer `var` bindings is rejected.
- `break` and `continue` are rejected.
- Indexed assignment and field assignment inside the loop body are rejected in v1.

Use `available_parallelism()` to inspect the runtime worker count.

---

break

- Immediately exits the innermost loop (`while` or `for`).
- Using it outside a loop causes a compile error.

```
for i in range(10):
    if i == 5:
        break     # Exits when i == 5
    print(i)      # 0 1 2 3 4
```

Error Example

```
# break outside a loop is a compile error
break    # Error: break outside loop
```

---

continue

- Ends the current iteration of the innermost loop and skips to the next iteration.
- Using it outside a loop causes a compile error.

```
for i in range(5):
    if i == 2:
        continue    # Skip i == 2
    print(i)        # 0 1 3 4
```

`...` (Ellipsis)

- A no-op statement that does nothing. Used as a placeholder for empty blocks.
- Can be used in any block: function body, `if`/`elif`/`else`, `while`, `for`, `match case`, etc.

```
fn not_yet():
    ...

if true:
    ...
else:
    ...
```

---

match

Syntax

```
match expression:
    case pattern:
        # body
    case pattern if guard_condition:
        # guarded body
    case _:
        # wildcard (matches anything)
```

Pattern Types

Pattern	Example	Description
Wildcard	`_`	Matches anything
Literal	`0`, `"hello"`, `true`	Equality comparison
Variable binding	`n`	Matches anything and binds to a variable
enum variant	`Color::Red`	Compares enum tag (simple enum)
ADT enum variant	`Shape::Circle(r)`	Matches an enum variant with associated data and binds it
`Some(x)`	`Some(v)`	When Option has a value, binds the inner value
`None`	`None`	When Option has no value
OR pattern	`1 | 2 | 3`	Matches if any alternative matches

Guard Clause

A guard condition can be specified in the form `case pattern if condition:`. The arm is executed only when the pattern matches and the guard condition is true.

OR Pattern

Multiple patterns can be combined with `|` to match any of them. Variable bindings (`n`, `Some(x)`, `Ok(v)`, `Err(e)`) are not allowed in OR patterns.

```
match x:
    case 1 | 2 | 3:
        print("small")
    case _:
        print("other")
```

```
# Enum OR pattern
match color:
    case Color::Red | Color::Blue:
        print("warm or cool")
    case Color::Green:
        print("green")
```

Exhaustiveness Checking

- enum types: Must cover all variants or include `_`. OR patterns count each alternative individually.
- Option types: Must cover both `Some` and `None` or include `_`.
- bool type: Must cover both `true` and `false` or include `_`.
- int / float / str literals: `_` is required.
- Guarded arms do not count toward exhaustiveness.

Example

```
# enum match
enum Color:
    Red
    Green
    Blue

match color:
    case Color::Red:
        print("red")
    case Color::Green:
        print("green")
    case Color::Blue:
        print("blue")

# Option match
let x: Option<int> = Some(42)
match x:
    case Some(v):
        print(v)
    case None:
        print("nothing")

# Literal match
match x:
    case 0:
        print("zero")
    case 1:
        print("one")
    case _:
        print("other")

# Guard clause
match x:
    case n if n > 0:
        print("positive")
    case n if n < 0:
        print("negative")
```

```
    case _:
        print("zero")
```

ADT Enum Match

When an enum variant carries associated data, use a binding pattern to extract the value(s).

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point

let s = Shape::Circle(3.14)
match s:
    case Shape::Circle(r):
        print(r)        # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")
```

Multi-field variants bind each field to a separate name in declaration order.

Scope Rules

- Each `case` arm has its own block scope.
- Variables bound by variable binding patterns (`n`) or `Some(x)` are only valid within that arm.

Scope Rules

Block Scope

- Each block of `if` / `elif` / `else` / `while` / `for` / `match` has a block scope.
- Variables declared inside a block go out of scope when the block ends.

```
for i in range(3):
    let tmp = i * 2
# tmp is not accessible here

if true:
    let a = 1
# a is not accessible here
```

Shadowing

- Declaring a variable with the same name as an outer variable in an inner scope causes the inner variable to be referenced within the inner scope.
- After leaving the inner scope, the outer variable is accessible again.

```
let x = 10
if true:
    let x = 99    # Shadows the outer x
    print(x)      # 99
print(x)          # 10
```

English | |

Directives

Directives are compile-time metadata annotations that can be attached to declarations. They use the `@name` syntax, similar to Java annotations.

Syntax

```
@name
@name(key=value, ...)
```

Directives are placed before the target declaration. Multiple directives can be stacked.

Supported Targets

Directives can be applied to the following declarations:

- `fn` - Function definitions
- `record` - Struct definitions
- `let` / `var` - Variable declarations
- Fields within a `record` definition
- `for` - Counted loops only for `@parallel`
- `it` - Test case definitions (for `@each` and `@property` only)

Built-in Directives

`@deprecated`

Marks a declaration as deprecated. When a deprecated entity is used (called, referenced, or accessed), a compile-time warning is emitted.

On functions:

```
@deprecated
fn old_function() -> int:
    return 42

print(old_function())   # warning: 'old_function' is deprecated
```

On types:

```
@deprecated
record OldPoint:
    x: int
    y: int

let p = OldPoint(1, 2)  # warning: 'OldPoint' is deprecated
```

On variables:

```
@deprecated
let old_value = 99

print(old_value)         # warning: 'old_value' is deprecated
```

On fields:

```
record Config:
    @deprecated
    old_setting: int
    new_setting: int
```

```
let c = Config(1, 2)
print(c.old_setting)     # warning: 'Config.old_setting' is deprecated
print(c.new_setting)     # no warning
```

`@native`

Declares a function whose implementation is provided by the runtime (built-in). The function must not have a body.

Basic syntax:

```
@native
fn contains(s: str, sub: str) -> bool

print(contains("hello world", "world"))  # true
```

Operator overloads:

```
@native
fn operator+(a: str, b: str) -> str

print("hello" + " world")  # hello world
```

UFCS-compatible:

```
@native
fn to_upper(s: str) -> str

print("hello".to_upper())  # HELLO
```

Argument count validation:

When a `@native` declaration includes a type signature, the compiler validates the number of arguments at call sites. Overloaded functions (e.g., `range` with 1, 2, or 3 arguments) are supported — any matching overload passes validation.

```
@native
fn range(n: int) -> List<int>
@native
fn range(start: int, end: int) -> List<int>

print(len(range(5)))       # OK: matches 1-arg overload
print(len(range(1, 10)))   # OK: matches 2-arg overload
print(len(range()))        # Error: expects 1 or 2 argument(s), but got 0
```

Standard library declarations (`core/`):

The `core/` directory contains `@native` declarations for all built-in functions, organized by category:

File	Contents
`core/builtins.ry`	`print`, `len`, `range`, `enumerate`, `zip`, `exit`, `args`, `available_parallelism`
`core/str.ry`	`contains`, `starts_with`, `ends_with`, `find`, `substring`, `char_at`, `replace`, `to_upper`, `to_lower`, `trim`, `trim_start`, `trim_end`, `repeat`, `reverse`, `split`, `join`
`core/convert.ry`	`to_int`, `to_float`, `to_str`

File	Contents
`core/list.ry`	`append`, `pop`, `insert`, `remove_at`, `slice`, `distinct`, `flatten`, `sort`, `first`, `last`, `is_empty`
`core/map.ry`	`keys`, `values`, `items`, `has_key`, `get`, `merge`
`core/set.ry`	`add`, `remove`, `union`, `intersection`, `difference`, `symmetric_difference`, `is_subset`, `is_superset`
`core/higher_order.ry`	`filter`, `map`, `reduce`, `fold`, `any`, `all`, `sum`, `min`, `max`

These files are automatically loaded as a prelude when the `core/` directory is found relative to the `ry` executable. The prelude enables argument count validation for built-in function calls.

Constraints: - `@native` functions must not have a body (no `:` after the signature). - Providing a body causes a parse error: `@native function must not have a body`. - The declared function must correspond to an existing built-in; otherwise the call will fail at compile time.

Future extensions: - `@native("libfoo.so")` — FFI binding to external shared libraries.

`@parallel`

Marks a counted `for` loop for parallel execution.

```
@parallel
for i in range(8):
    work(i)
```

Supported target:

- `for` statements only

Constraints:

- Only a single `@parallel` directive is allowed on a `for` statement.
- The iterable must be `range(...)` or an integer `..` range.
- Destructuring iteration is not supported.
- Assigning to outer mutable variables is rejected.
- `break`, `continue`, indexed assignment, and field assignment inside the loop body are rejected in v1.

`@each`

Enables parameterized testing by running an `it` block multiple times with different parameters.

Syntax:

```
@each([(arg1, arg2, ...), ...])
it("description with {0} and {1}", fn(param1: type, param2: type):
    # test body
)
```

Supported target: `it` calls only

Constraints: - The argument must be a list of tuples - Tuple arity must match the lambda parameter count - Placeholders `{0}`, `{1}`, … in the description string are replaced with stringified values

`@property`

Enables property-based testing by generating random inputs for an `it` block.

Syntax:

```
@property(count=100)
it("property name", fn(a: int, b: int):
    # test body with random values
)
```

Supported target: `it` calls only

Parameters:

Parameter	Type	Default	Description
`count`	int	100	Number of random trials

Supported parameter types:

Type	Range
`int`	-1000 to 1000
`float`	-1000.0 to 1000.0
`bool`	true or false
`str`	Random ASCII, 0-20 characters

On failure, the counterexample (parameter values that caused the failure) is printed.

Parameters (future extension)

Directives support an optional parameter syntax for future extensions:

```
@deprecated(reason="use new_api instead")
fn old_api() -> int:
    return 0
```

Currently, parameters are parsed but not used by the `@deprecated` directive.

Notes

- Deprecated entities still function normally; only a warning is emitted.
- Warnings are emitted at the point of use, not at the definition.
- Defining a deprecated entity without using it produces no warnings.
- Unknown directive names cause a parse error.
- Directives on unsupported targets (e.g., `if`, `while`) cause a parse error. `@parallel` is the only directive supported on `for`.

English | |

Error Reference

Error Format

ry displays compile errors in a Rust-inspired rich format that shows the exact location of the error with source context:

```
error: cannot reassign let variable: x
  --> main.ry:5:1
  |
5 | x = 10
  | ^ cannot reassign let variable: x
```

Each error message includes: - **Error level and message** (`error: ...`) - **File location** (`--> file:line:col`) - **Source line** with the relevant code - **Caret indicator** (^) pointing to the exact column

Compile Errors

Error	Cause	Example
Assignment to undeclared variable	Assigned to a variable that has not been declared	`x = 1` (`x` is undeclared)
Reassignment to let variable	Reassigned to a variable declared with `let`	`let x = 1` -> `x = 2`
Redeclaration of same-named variable	Redeclared a variable with the same name in the same scope	`let x = 1` -> `let x = 2`
Type-changing reassignment	Assigned a value of a different type to a variable	`let x = 1` -> `x = 3.14`
Type annotation mismatch	Declared type and assigned value type differ	`let x: int = 3.14`
Overload return type conflict	Defined overloads with the same parameter types but different return types only	Two functions with parameters `(int, int)` returning `int` and `float`
Overload resolution failure	No overload matches the argument types	`fn add(a: int, b: int)` called with `add(1.0, 2.0)`
Float in bitwise operation	Passed `float` type to `&`, |, `^`, `~`, `<<`, `>>`	`3.14 & 1`
Empty list	Cannot infer type from empty list literal `[]`	`let xs = []`
Empty map	Cannot infer type from empty map literal `{}`	`let m = {}`
Tuple out-of-range index	Accessed a non-existent index on a tuple	`let t = (1, 2)` -> `t.2`
break/continue outside loop	Used `break` or `continue` outside a `for`/`while` loop	`break` at function top level
Module import inside block	Used `from` statement inside a function or conditional block	`from math` inside a function
Circular import	Modules import each other	`a.ry` imports `b.ry` and `b.ry` imports `a.ry`
Duplicate field name	Defined the same field name twice in a struct	Defining `x` twice in `type T: x: int`
Non-exhaustive match	match does not cover all patterns	Some enum variants uncovered, missing `None` for Option, no `_` for literals
`!!` return type mismatch	`!!` used in function not returning `(T, Error?)`	Using `!!` in a function returning `int`
`!!` operand type mismatch	`!!` applied to non `(T, Error?)` value	Applying `!!` to a plain `int`

Compile Error Examples

```
# Reassignment to let variable
let x = 10
x = 20    # Error

# Type-changing reassignment
let n = 1
n = "hello"    # Error: assigning str to int variable

# Empty list
let xs = []    # Error: type cannot be inferred

# break outside loop
break    # Error: outside loop

# Import inside block
fn foo():
    from math    # Error: top level only

# Duplicate field name
record Bad:
    x: int
    x: float    # Error: x is duplicated
```

Runtime Errors

Error	Cause	Example
List out-of-range access	List index exceeds bounds	`let xs = [1, 2, 3] -> xs[5]`
Map non-existent key access	Referenced a key that does not exist in the map	`let m = {"a": 1} -> m["b"]`
Contract violation	A `require`, `ensure`, or `invariant` condition evaluated to false	See Design by Contract

All runtime errors terminate the process with `exit(1)`.

Runtime Error Examples

```
# List out-of-range access
let xs = [1, 2, 3]
print(xs[10])    # Runtime error: exit(1)

# Map non-existent key access
let m = {"a": 1}
print(m["z"])    # Runtime error: exit(1)
```

English | |

Function Reference

Function Definition Syntax

```
fn function_name(param_name: type, ...) -> return_type:
    # body
    return value
```

- Parameter types are required.
- Return type is optional (defaults to `Unit` when omitted).
- The body is an indented block.
- Functions can have `require` (precondition) and `ensure` (postcondition) clauses. See Design by Contract.

  Naming convention: Function names and parameter names must use snake_case (e.g., `add`, `get_value`, `map_list`). The compiler enforces this convention.

```
fn add(a: int, b: int) -> int:
    return a + b

fn greet(name: str):
    print("Hello, " + name)    # Return type is Unit
```

Parameter and Return Types

Item	Description
Parameter type	Required. All parameters must have type annotations
Return type	Optional. Defaults to `Unit` (equivalent to void) when omitted
`Unit`	Return type for functions that return no value

```
fn no_return(x: int):       # Return type Unit (omitted)
    print(x)

fn get_value() -> int:      # Return type int
    return 42
```

Recursion

Functions can call themselves.

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

Overloading

Multiple functions with the same name can be defined if they differ in the number or types of parameters.

Rules

- Functions with the same name can be defined if the number or types of parameters differ.
- The appropriate function is selected at the call site based on the argument types and count.
- Overloading by return type alone is not allowed.

```
fn area(side: int) -> int:
    return side * side

fn area(w: int, h: int) -> int:
    return w * h

let a = area(5)       # 25
let b = area(3, 4)    # 12
```

Unit Type Functions

Functions without a return value return `Unit`. The return type can be omitted.

```
fn log(msg: str):
    print(msg)

fn log_typed(msg: str) -> Unit:
    print(msg)
```

Tasks And Async Functions

`Task<T>` is the built-in handle type for concurrent work. `async fn` returns `Task<T>`, `await` extracts `T`, and `join(task)` is the blocking function-form equivalent of `await task`.

```
async fn add(a: int, b: int) -> int:
    return a + b

let t: Task<int> = add(20, 22)
print(await t)          # 42
await add(1, 2)         # waits and discards the result
print(join(add(1, 2)))  # 3
```

Rules

- `async fn name(...) -> T:` is declared with the awaited result type `T`.
- Calling an `async fn` immediately returns `Task<T>`.
- `await expr` requires `expr` to be `Task<T>` and produces `T`.
- `await` is allowed anywhere an expression is allowed, and `await expr` is also allowed as a statement.
- `async fn ... -> Unit` is supported; `await task` is the primary way to wait when no value is produced.
- Tasks run on the runtime worker pool; they are not implemented as one OS thread per task.
- `async` lambdas and `async @native fn` are not supported in v1.

`Channel<T>` is the built-in handle type for blocking message passing between tasks. Create channels with `channel[T]()` or `channel[T](capacity)`, send values with `send(ch, value)`, use `try_send(ch, value)` for a non-blocking send attempt, use `recv(ch)` for strict receive, use `recv_opt(ch)` for close-aware receive, use `try_recv(ch)` for a non-blocking receive attempt, iterate consumers with `for x in ch:`, and terminate a channel with `close(ch)`.

Lambda Functions

Anonymous functions can be defined inline.

Syntax

```
# Single expression (the expression value is returned; return type is inferred)
fn(param_name: type, ...): expression

# Multi-line block
fn(param_name: type, ...):
    # multiple statements
    return value

# With explicit return type (optional)
fn(param_name: type, ...) -> return_type: expression
```

Example

```
let double = fn(x: int): x * 2
let result = double(5)    # 10

let add = fn(a: int, b: int): a + b
let sum = add(3, 4)       # 7

# Multi-line lambda
let abs = fn(x: int):
    if x < 0:
        return -x
    return x
```

Closures

Lambda functions **capture by value** the variables from the outer scope at the time of definition.

```
let base = 10
let add_base = fn(x: int): x + base    # Captures base by value

base = 99           # Does not affect the captured value
let r = add_base(5)    # 15 (uses base = 10 from capture time)
```

Capture Rules

Item	Details
Capture method	Capture by value (copy)
Capture timing	At lambda definition time
Effect of outer variable changes	None (because it is a copy)

Function Type

A type for treating functions as values.

Syntax

```
fn(param_type1, param_type2, ...) -> return_type
```

Example

```
let f: fn(int) -> int = fn(x: int): x * 2
let g: fn(int, int) -> int = fn(a: int, b: int): a + b

fn apply(func: fn(int) -> int, x: int) -> int:
    return func(x)

let result = apply(f, 5)   # 10
```

Higher-Order Functions

Functions can accept functions as arguments or return them as values.

```
fn map_list(xs: List<int>, f: fn(int) -> int) -> List<int>:
    let result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

let doubled = map_list([1, 2, 3], fn(x: int): x * 2)
# [2, 4, 6]
```

UFCS (Uniform Function Call Syntax)

`a.f(b)` can be used to call `f(a, b)`. Useful for method chaining.

Syntax

```
# Normal call
f(a, b)

# UFCS call (equivalent)
a.f(b)
```

Chaining

```
fn double(x: int) -> int:
    return x * 2

fn add_one(x: int) -> int:
    return x + 1

let result = 5.double().add_one()   # double(5) -> 10, add_one(10) -> 11
```

Mixing with Field Access

Field access (`.field`) and UFCS (`.method()`) use the same dot notation but are distinguished by the presence of arguments.

```
let p = Point(3, 4)
let len = p.x.to_float()    # Field access + UFCS
```

Operator Overloading

You can define operator behavior for user-defined types.

Syntax

```
# Binary operator (2 parameters)
fn operator<op>(a: type, b: type) -> return_type:
    ...

# Unary operator (1 parameter)
fn operator<op>(a: type) -> return_type:
    ...
```

Overloadable Operators

Category	Operators
Arithmetic (binary)	`+ - * / % ** //`
Comparison (binary)	`== != < <= > >=`
Bitwise (binary)	`& | ^ << >>`
Logical (binary)	`and or`
Unary	`- ~ not`

Distinguishing Binary and Unary

Distinguished by the number of parameters.

```
record Vec2:
    x: float
    y: float

# Binary +
fn operator+(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x + b.x, a.y + b.y)

# Unary -
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)

# Comparison
fn operator==(a: Vec2, b: Vec2) -> bool:
    return a.x == b.x and a.y == b.y

let v1 = Vec2(1.0, 2.0)
let v2 = Vec2(3.0, 4.0)
let v3 = v1 + v2     # Vec2(4.0, 6.0)
let v4 = -v1         # Vec2(-1.0, -2.0)
```

English | |

I/O Function Reference

Standard I/O and file operations. All functions require explicit import from `std.io`.

```
from std.io import read_text, write_text, file_exists
```

Function List

Standard Input

Function	Signature	Description
`read_line`	`() -> str`	Reads one line from stdin (trailing newline removed)
`read_all`	`() -> str`	Reads all of stdin until EOF

File I/O

Function	Signature	Description
`read_text`	`(str) -> str`	Reads entire file as a string
`write_text`	`(str, str) -> Unit`	Writes a string to a file (overwrites)
`append_text`	`(str, str) -> Unit`	Appends a string to the end of a file
`file_exists`	`(str) -> bool`	Checks if a file exists
`delete_file`	`(str) -> Unit`	Deletes a file
`read_bytes`	`(str) -> List<byte>`	Reads a file as a byte list
`write_bytes`	`(str, List<byte>) -> Unit`	Writes a byte list to a file

Byte Conversions

Function	Signature	Description
`str_to_bytes`	`(str) -> List<byte>`	Converts a string to UTF-8 bytes
`bytes_to_str`	`(List<byte>) -> str`	Converts a byte list to a string

Examples

Reading and Writing Files

```
from std.io import read_text, write_text, append_text, file_exists, delete_file

write_text("hello.txt", "Hello, World!")
let content = read_text("hello.txt")
print(content)   # Hello, World!

append_text("hello.txt", "\nGoodbye!")
print(read_text("hello.txt"))
# Hello, World!
# Goodbye!

print(file_exists("hello.txt"))   # true
delete_file("hello.txt")
print(file_exists("hello.txt"))   # false
```

Byte Operations

```
from std.io import str_to_bytes, bytes_to_str, write_bytes, read_bytes

let bs = str_to_bytes("ABC")
print(len(bs))     # 3

write_bytes("data.bin", bs)
let rb = read_bytes("data.bin")
let s = bytes_to_str(rb)
print(s)           # ABC
```

Reading from Standard Input

```
from std.io import read_line

let name = read_line()
print(f"Hello, {name}!")
```

Error Handling

All file operations terminate with a runtime error if the operation fails:

Operation	Error Condition
`read_text` / `read_bytes`	File does not exist or cannot be opened
`write_text` / `write_bytes` / `append_text`	File cannot be opened for writing
`delete_file`	File cannot be deleted

Error messages are printed to stderr and the program exits with code 1.

Notes

- `List<byte>` is used as the buffer type. Standard list operations (`len()`, `append()`, `slice()`, index access) all work with byte lists.
- File paths are relative to the current working directory unless absolute paths are specified.
- `write_text` and `write_bytes` overwrite existing files. Use `append_text` to add content to existing files.

English | |

Math Functions (`std.math`)

Overview

The `std.math` package provides mathematical constants and functions. Unlike the `std` package, it is not automatically imported. Use explicit import to access the functions.

```
from std.math import sqrt, PI, sin
```

Constants

Constant	Type	Description
`PI`	`float`	Pi (3.141592653589793)
`E`	`float`	Euler's number (2.718281828459045)
`Inf`	`float`	Positive infinity
`NaN`	`float`	Not a Number

```
from std.math import PI, E, Inf, NaN

let circumference = 2.0 * PI * radius
```

Absolute Value

Function	Signature	Description
`abs(x)`	`(int) -> int`	Absolute value of integer
`abs(x)`	`(float) -> float`	Absolute value of float

```
from std.math import abs

abs(-5)       # 5
abs(-3.14)    # 3.14
```

Rounding

Function	Signature	Description
`floor(x)`	`(float) -> int`	Round down to nearest integer
`ceil(x)`	`(float) -> int`	Round up to nearest integer
`round(x)`	`(float) -> int`	Round to nearest integer

```
from std.math import floor, ceil, round

floor(3.7)    # 3
ceil(3.2)     # 4
round(3.5)    # 4
```

Power and Root

Function	Signature	Description
`sqrt(x)`	`(float) -> float`	Square root
`pow(x, y)`	`(float, float) -> float`	x raised to the power of y

```
from std.math import sqrt, pow

sqrt(9.0)       # 3.0
pow(2.0, 3.0)   # 8.0
```

Logarithm

Function	Signature	Description
`log(x)`	`(float) -> float`	Natural logarithm (base e)
`log2(x)`	`(float) -> float`	Base-2 logarithm
`log10(x)`	`(float) -> float`	Base-10 logarithm

Exponential

Function	Signature	Description
`exp(x)`	`(float) -> float`	e raised to the power of x

Trigonometric

Function	Signature	Description
`sin(x)`	`(float) -> float`	Sine (x in radians)
`cos(x)`	`(float) -> float`	Cosine (x in radians)
`tan(x)`	`(float) -> float`	Tangent (x in radians)

Inverse Trigonometric

Function	Signature	Description
`asin(x)`	`(float) -> float`	Arc sine (result in radians)
`acos(x)`	`(float) -> float`	Arc cosine (result in radians)
`atan(x)`	`(float) -> float`	Arc tangent (result in radians)

Two-argument Functions

Function	Signature	Description
`atan2(y, x)`	`(float, float) -> float`	Arc tangent of y/x (result in radians)
`hypot(x, y)`	`(float, float) -> float`	Hypotenuse: sqrt(x^2 + y^2)

Predicates

Function	Signature	Description
`is_nan(x)`	`(float) -> bool`	True if x is NaN

Function	Signature	Description
`is_inf(x)`	`(float) -> bool`	True if x is positive or negative infinity

```
from std.math import is_nan, is_inf, NaN, Inf

is_nan(NaN)    # true
is_inf(Inf)    # true
is_nan(1.0)    # false
```

English | |

Operator Reference

Precedence Table

Lower numbers indicate higher precedence (evaluated first).

Precedence	Operator	Description	Associativity
0	`!!`	Error propagation (postfix)	Left
1	`()`	Grouping	–
2	`+x -x ~x`	Unary plus, unary minus, bitwise NOT	Right
3	`**`	Exponentiation	Right
3.5	`as`	Type cast	Left
4	`* / % //`	Multiplication, division, modulo, integer division	Left
5	`+ -`	Addition, subtraction	Left
6	`<< >> >>>`	Bit shift	Left
7	`&`	Bitwise AND	Left
8	`^`	Bitwise XOR	Left
9	`|`	Bitwise OR	Left
10	`== != < <= > >= in not in`	Comparison, membership	Left
11	`not`	Logical NOT	Right
12	`and`	Logical AND	Left
13	`or`	Logical OR	Left
13.5	`??`	Null coalescing	Left
14	`?:`	Ternary conditional	Right

Arithmetic Operators

Operator	Description	Example
`+`	Addition / string concatenation	`1 + 2 -> 3, "a" + "b" -> "ab"`
`-`	Subtraction	`5 - 3 -> 2`
`*`	Multiplication / string repetition	`4 * 3 -> 12, "ab" * 3 -> "ababab"`
`/`	Division (always float)	`7 / 2 -> 3.5`
`//`	Integer division (truncated)	`7 // 2 -> 3`
`%`	Modulo	`7 % 3 -> 1`
`**`	Exponentiation (always float)	`2 ** 10 -> 1024.0`

Operator	Description	Example
`-x`	Unary minus	`-5`, `-3.14`
`+x`	Unary plus	`+5` (no sign change)

```
let a = 10 // 3    # 3 (int)
let b = 10 / 3     # 3.3333... (float)
let c = 2 ** 8     # 256.0 (float)
let s = "foo" + "bar"   # "foobar"
```

Comparison Operators

All return `bool`.

Operator	Description
`==`	Equal
`!=`	Not equal
`<`	Less than
`<=`	Less than or equal
`>`	Greater than
`>=`	Greater than or equal

- Can be used with numeric types (int / float) and bool.
- `str` values are compared lexicographically (byte order).
- The `in` operator is used for membership checks on sets, lists, and maps (`x in s`).
- The `not in` operator is the negation of `in` (`x not in s`).
- For maps, `in` checks whether the key exists.

```
let x = 3 < 5        # true
let y = "abc" < "abd"   # true (lexicographic)
let s = {1, 2, 3}
let z = 2 in s       # true
let w = 4 not in s   # true
let xs = [1, 2, 3]
let a = 2 in xs      # true (list linear search)
let m = {"a": 1}
let b = "a" in m     # true (map key lookup)
```

Logical Operators

Operator	Description	Type
`and`	Logical AND	`bool` x `bool` -> `bool`
`or`	Logical OR	`bool` x `bool` -> `bool`
`not`	Logical NOT	`bool` -> `bool`

```
let a = true and false   # false
let b = true or false    # true
let c = not true         # false
```

Bitwise Operators

Only available for `int` type. Applying to `float` or `bool` causes a compile error.

Operator	Description	Example
`&`	Bitwise AND	`0b1100 & 0b1010 -> 0b1000`
`|`	Bitwise OR	`0b1100 | 0b1010 -> 0b1110`
`^`	Bitwise XOR	`0b1100 ^ 0b1010 -> 0b0110`
`~`	Bitwise NOT (unary)	`~0 -> -1`
`<<`	Left shift	`1 << 4 -> 16`
`>>`	Arithmetic right shift	`16 >> 2 -> 4`
`>>>`	Logical right shift	`-1 >>> 1 -> 9223372036854775807`

```
let flags = 0b0001 | 0b0010   # 3
let masked = flags & 0b0011   # 3
let shifted = 1 << 8          # 256
```

Ternary Conditional Operator

```
let x = condition ? true_value : false_value
```

Evaluates `condition`. If truthy, returns `true_value`; otherwise returns `false_value`. Both branches must have the same type. Right-associative, so nested ternaries associate from right to left.

```
let x = 3 > 2 ? 10 : 20      # 10
let s = false ? "yes" : "no" # "no"

# Nested (right-associative)
let y = true ? (false ? 1 : 2) : 3   # 2
```

Range Operator

The `..` operator creates an inclusive integer range.

```
let xs = 1 .. 5     # [1, 2, 3, 4, 5]

for i in 1 .. 3:
    print(i)        # 1 2 3
```

The result is a `List<int>` containing all integers from the left operand to the right operand (inclusive).

Null Coalescing Operator (??)

```
let x = option_val ?? default_val
```

If `option_val` is `Some(v)`, returns `v`. Otherwise returns `default_val`. The right-hand operand must have the same type as the inner type of the Option.

```
let a: int? = Some(10)
let b: int? = none

print(a ?? 0)     # 10
print(b ?? 0)     # 0
```

Compound Assignment Operators

Shorthand for updating a variable. `x op= y` is equivalent to `x = x op y`.

Operator	Equivalent Expression
`x += y`	`x = x + y`
`x -= y`	`x = x - y`
`x *= y`	`x = x * y`
`x /= y`	`x = x / y`
`x %= y`	`x = x % y`
`x //= y`	`x = x // y`
`x **= y`	`x = x ** y`
`x &= y`	`x = x & y`
`x |= y`	`x = x | y`
`x ^= y`	`x = x ^ y`
`x <<= y`	`x = x << y`
`x >>= y`	`x = x >> y`

```
var x = 10
x += 5     # x = 15
x -= 3     # x = 12
x *= 2     # x = 24
x //= 3   # x = 8
x &= 6    # x = 0
```

Increment / Decrement Operators

Postfix-only, statement-level operators for incrementing or decrementing a variable by 1. These are desugared to `x = x + 1` and `x = x - 1` respectively.

Operator	Equivalent Expression
`x++`	`x = x + 1`
`x--`	`x = x - 1`

```
var count = 0
count++        # count = 1
count++        # count = 2
count--        # count = 1

var f = 1.5
f++            # f = 2.5 (int 1 is promoted to float)
```

> **Note**: `++` / `--` can only be used as statements, not as expressions. `let` variables cannot be incremented/decremented (immutability is enforced).

Type Rules for Operations

Operation	Left Type	Right Type	Result Type
`+ - *`	int	int	int
`+ - *`	float	int / float	float

Operation	Left Type	Right Type	Result Type
`+ - *`	int	float	float
`/`	any numeric	any numeric	float
`//`	any numeric	any numeric	int
`**`	any numeric	any numeric	float
`%`	int	int	int
`%`	float or int (one is float)	–	float
`+`	str	str	str
`== != < <= > >=`	numeric / bool / str	same type	bool
`*`	str	int	str
`in`	any	Set / List / Map<K, V>	bool
`not in`	any	Set / List / Map<K, V>	bool
`& | ^ ~ << >> >>>`	int	int	int
`and or not`	bool	bool	bool

Operator Overloading

You can define operator behavior for user-defined types.

Syntax

```
# Binary operator (2 parameters)
fn operator+(a: MyType, b: MyType) -> MyType:
    ...

# Unary operator (1 parameter)
fn operator-(a: MyType) -> MyType:
    ...
```

Overloadable Operators

Category	Operators
Arithmetic (binary)	`+ - * / % ** //`
Comparison (binary)	`== != < <= > >=`
Bitwise (binary)	`& | ^ << >> >>>`
Logical (binary)	`and or`
Unary	`- ~ not`

Distinguishing Binary and Unary

Distinguished by the number of parameters.

```
# Binary -
fn operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)

# Unary -
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

English | |

Package Reference

Overview

Ry uses a package system to organize code. A **package** can be either a single `.ry` file or a directory containing multiple `.ry` files. Use the `from` statement to import packages.

The `std` package (standard library) is automatically imported into every program.

Import Syntax

Import All Definitions

```
from math
```

Imports all functions and types from the package.

Selective Import

```
from math import add
```

Imports only the specified definition.

Multiple Selective Import

```
from math import add, sub
```

Imports multiple definitions separated by commas.

Package Resolution

Packages are resolved using dot-separated notation:

Import Statement	Resolution
`from math`	`math/` directory (package) or `math.ry` file
`from utils.math`	`utils/math/` directory or `utils/math.ry` file
`from std.str`	`std/str/` directory or `std/str.ry` file

Resolution Order

For each search path, the system checks: 1. **Directory** (`{path}/`) — if exists, all `.ry` files in the directory are loaded (package) 2. **File** (`{path}.ry`) — single file (backward compatible)

Directory Packages

When a package resolves to a directory: - All `.ry` files in the directory are automatically loaded - Files starting with `_` are excluded - No special entry file (like `__init__.py`) is needed - All functions and types defined in the directory's files are exported

Private Symbols

Definitions whose names start with `_` (underscore) are private to the package and cannot be imported:

- Wildcard imports (`from pkg`) automatically exclude `_`-prefixed symbols
- Named imports (`from pkg import _helper`) produce a compile error

```
# mylib/internal.ry
fn _helper() -> int:      # private - not importable
    return 42
fn public_api() -> int:  # public - importable
    return _helper()
```

```
mypackage/
  math.ry      # fn add(), fn sub()
  string.ry    # fn concat()
```

```
from mypackage          # imports add, sub, concat
from mypackage import add   # imports only add
```

Standard Library (std)

The `std` package is automatically imported into every program. It provides: - Built-in functions (`print`, `len`, `range`, etc.) - String functions (`contains`, `find`, `replace`, etc.) - Type conversion functions (`to_int`, `to_float`, `to_str`) - Collection functions (`map`, `filter`, `sort`, etc.)

Sub-packages

The following sub-packages require explicit import:

Package	Description
`std.math`	Mathematical constants and functions
`std.io`	File I/O, standard input, and byte conversions

```
from std.math import sqrt, PI, sin
```

You can also explicitly import specific definitions from `std`:

```
from std.str import contains
```

RY_HOME

The standard library is installed at `$RY_HOME/lib/std/`. The default value of `RY_HOME` is `~/.ry`.

```
export RY_HOME="$HOME/.ry"    # default
```

Search Path Priority

1. The directory of the importing file
2. `$RY_HOME/lib` (standard library location)
3. Executable-relative `lib/` directories
4. Paths specified in the `RY_PATH` environment variable (colon-separated)

RY_PATH Environment Variable

Specifying colon-separated directories in `RY_PATH` adds them to the package search path.

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

Constraints

Constraint	Details
Allowed location	Top level only (not inside functions or blocks)
Duplicate imports	Automatically skipped (no error)
Circular imports	Compile error

```
# Error example: Import inside a block
fn main():
    from math   # Error: imports only allowed at top level

# OK: Importing the same package multiple times does not cause an error
from math
from math   # Skipped
```

Creating Package Files

Single File Package

```
# math.ry
fn add(a: int, b: int) -> int:
    return a + b

fn sub(a: int, b: int) -> int:
    return a - b
```

```
# main.ry
from math import add, sub

print(add(1, 2))   # 3
print(sub(5, 3))   # 2
```

Directory Package

```
mylib/
  math.ry
  string.ry
```

```
# main.ry
from mylib import add, concat
```

English | |

Project Management

ry init - Project Initialization

Initializes the current directory as a Ry project.

```
ry init
```

Generated Files and Directories

```
my-project/
  ry.toml          # Project configuration file
```

```
  src/
    main.ry        # Entry point (sample code)
```

Behavior

1. Exits with an error if `ry.toml` already exists
2. Creates the `src/` directory (if it doesn't exist)
3. Generates `ry.toml` (`name` is set to the current directory name)
4. Generates `src/main.ry` (skipped if it already exists)

`ry self-update` - Self Update

Updates ry itself to the latest version. Downloads a binary from GitHub Releases and replaces the current executable.

```
ry self-update                # Update to the latest stable version
ry self-update --nightly      # Update to the latest nightly pre-release
ry self-update v0.0.1         # Update to a specified version
```

Behavior

1. Displays the current version
2. Resolves the target version based on arguments
   - No arguments: Latest stable release from GitHub (`/releases/latest`)
   - `--nightly`: Latest pre-release
   - Version specified: Release with the specified tag
3. If the current version is the same, exits with `"Already up to date."`
4. Downloads the binary and replaces the current executable

Notes

- Requires `curl` and `tar` commands
- If replacing the binary fails due to insufficient permissions, a message suggesting `sudo` is displayed (sudo is not invoked automatically)
- Downloads are performed to a temporary directory first; however, if the cross-filesystem `cp` fallback is interrupted, the destination binary may be left in a partial state

---

`ry.toml` Configuration File

Describes project metadata and path settings in TOML format.

```toml
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"

[paths]
src = "src"
```

`[project]` Section

Key	Description
name	Project name (directory name at initialization)
version	Version string
entry	Source file serving as the entry point

[paths] Section

Key	Description
src	Source code directory

TOML Subset Specification

ry.toml supports the following TOML subset.

- Section headers: [section]
- Key-value pairs: key = "value" (string values only)
- Comments: From # to end of line
- Blank lines are ignored

English | |

Regular Expression Function Reference

A list of regular expression functions. All functions support UFCS notation. Pattern strings use standard regex syntax.

> **Note:** Phase 1 passes patterns as plain strings. A dedicated regex literal syntax may be added in the future.

Function List

Function	Signature	Description
regex_match	(str, str) -> bool	Returns whether the entire text matches the pattern
regex_search	(str, str) -> int	Returns the start position of the first match (-1 if not found)
regex_replace	(str, str, str) -> str	Replaces all matches with a replacement string
regex_split	(str, str) -> List<str>	Splits text by pattern matches
regex_find_all	(str, str) -> List<str>	Returns all non-overlapping matches

Supported Pattern Syntax

Syntax	Description	Example
abc	Literal characters	"hello"
.	Any character (except newline)	"a.c" matches "abc", "aXc"
|	Alternation	“cat|dog” matches "cat" or "dog"
*	Zero or more	"a*" matches "", "a", "aaa"
+	One or more	"a+" matches "a", "aaa"

Syntax	Description	Example
?	Zero or one	"a?" matches "" or "a"
{n}	Exactly n times	"a{3}" matches "aaa"
{n,m}	Between n and m times	"a{2,4}" matches "aa" to "aaaa"
{n,}	At least n times	"a{2,}" matches "aa", "aaa", …
*?	Zero or more (non-greedy)	".*?" matches shortest
+?	One or more (non-greedy)	".+?" matches shortest
??	Zero or one (non-greedy)	"a??" prefers zero
{n,m}?	Range (non-greedy)	"a{2,4}?" prefers n times
(...)	Group	"(ab)+" matches "abab"
[abc]	Character class	"[aeiou]" matches vowels
[a-z]	Character range	"[a-z]+" matches lowercase words
[^...]	Negated character class	"[^0-9]" matches non-digits
^	Start of string anchor	"^hello"
$	End of string anchor	"world$"
\d	Digit [0-9]	"\d+" matches numbers
\D	Non-digit [^0-9]	
\w	Word character [a-zA-Z0-9_]	"\w+" matches identifiers
\W	Non-word character	
\s	Whitespace	"\s+" matches spaces/tabs
\S	Non-whitespace	
\b	Word boundary	"\bword\b" matches whole word
\B	Non-word boundary	"\Bword" matches inside a word
(?i)	Case-insensitive flag	"(?i)hello" matches "HELLO"
\.	Escaped special character	"\." matches literal .

Usage Examples

regex_match

```
print(regex_match("[a-z]+", "hello"))   # true
print(regex_match("[0-9]+", "hello"))   # false
print(regex_match("[a-zA-Z_]\\w*", "my_var"))  # true
```

regex_search

```
let pos = regex_search("[0-9]+", "abc123def")
print(pos)  # 3
```

regex_replace

```
let s = regex_replace("[0-9]+", "a1b2c3", "X")
print(s)  # aXbXcX
```

regex_split

```
let parts = regex_split("\\s+", "hello  world  foo")
print(len(parts))  # 3
print(parts[0])    # hello
```

regex_find_all

```
let matches = regex_find_all("[0-9]+", "a1b23c456")
print(len(matches))  # 3
print(matches[0])    # 1
print(matches[1])    # 23
print(matches[2])    # 456
```

Range Quantifiers

```
print(regex_match("\\d{3}-\\d{4}", "123-4567"))  # true
print(regex_match("a{2,4}", "aaa"))               # true
print(regex_match("(ab){2,}", "ababab"))           # true
```

Non-Greedy (Lazy) Match

```
# Greedy: matches longest
let g = regex_replace("\".*\"", "\"a\" and \"b\"", "X")
print(g)  # X

# Non-greedy: matches shortest
let l = regex_replace("\".*?\"", "\"a\" and \"b\"", "X")
print(l)  # X and X

# Find individual HTML-like tags
let tags = regex_find_all("<.*?>", "<a> <bb> <ccc>")
print(len(tags))  # 3
```

> **Note:** Non-greedy matching controls the overall match length. Without support for extracting parenthesized groups, mixed greedy/lazy patterns may behave differently from PCRE-style engines.

Word Boundary

```
# Match whole words only
let pos = regex_search("\\bworld\\b", "hello world")
print(pos)  # 6

# Find all words
let words = regex_find_all("\\b\\w+\\b", "hello world foo")
print(len(words))  # 3

# \B matches non-boundary (inside a word)
let pos2 = regex_search("\\Bworld", "helloworld")
print(pos2)  # 5
```

Case-Insensitive Matching

```
# (?i) at the start of pattern enables case-insensitive matching
print(regex_match("(?i)hello", "HELLO"))  # true
print(regex_match("(?i)hello", "Hello"))  # true

# Works with character classes
print(regex_match("(?i)[a-z]+", "ABC"))  # true

# Works with replace and find_all
```

```
let s = regex_replace("(?i)hello", "Hello HELLO hello", "X")
print(s)  # X X X
```

> **Note:** `(?i)` must appear at the beginning of the pattern and applies to the entire pattern. Partial case-insensitive matching (e.g., `(?i:sub)pattern`) is not supported.

UFCS Notation

```
# pattern.function(text, ...)
let m = "[a-z]+".regex_match("hello")
print(m)  # true
```

English | |

Struct Reference

Overview

Structs are value types allocated on the stack. They are defined with the `record` keyword. Structs can have `invariant` clauses for Design by Contract. See Design by Contract.

> **Naming convention**: Struct names must use PascalCase (e.g., `Point`, `Rectangle`). Field names must use snake_case. The compiler enforces these conventions.

Definition Syntax

```
record TypeName:
    field_name: type
    field_name: type
```

Example

```
record Point:
    x: int
    y: int

record Rectangle:
    width: float
    height: float
```

Constructor

Arguments are passed in the order of field definitions. Named arguments are not supported.

```
let p = Point(10, 20)
let r = Rectangle(3.0, 4.5)
```

Field Access

Fields are read using dot notation.

```
let p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

Field Assignment

Variable Declaration	Field Assignment
`var`	Allowed
`let`	Compile error

```
var p = Point(10, 20)
p.x = 100     # OK: var variable

let q = Point(10, 20)
q.x = 100     # Error: fields of let variables cannot be modified
```

Usage as Function Parameters and Return Values

```
fn distance(p: Point) -> float:
    return (p.x * p.x + p.y * p.y) as float

fn make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

Nested Structs

Structs can be used as fields of other structs.

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

let c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

Constraints and Errors

Constraint	Details
Duplicate field names	Compile error
Field assignment on `let` variables	Compile error
Passing a struct directly to `print`	Compile error (not supported by print)

```
# Error example: Duplicate field names
record Bad:
    x: int
    x: int   # Error

# Error example: Passing a struct to print
let p = Point(1, 2)
print(p)   # Error
```

Enumerations (enum)

Overview

Enumerations are a set of named constants. Internally, they are represented as i64 integers (0, 1, 2, …).

Definition Syntax

```
enum TypeName:
    VariantName
    VariantName
    ...
```

Example

```
enum Color:
    Red
    Green
    Blue
```

Variant Access

Variants are accessed using the `::` operator.

```
let c = Color::Red
print(c)   # Red
```

Comparison

Since enum values are integers, they can be compared directly with `==` / `!=`.

```
print(Color::Red == Color::Red)     # true
print(Color::Red != Color::Green)   # true
```

Usage in if Statements

```
let c = Color::Green
if c == Color::Red:
    print("red")
elif c == Color::Green:
    print("green")
else:
    print("blue")
```

Function Parameters

Use the enum name as the type name.

```
fn is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

print

`print()` outputs the variant name.

```
let c = Color::Blue
print(c)   # Blue
```

Constraints and Errors

Constraint	Details
Variant access requires `EnumName::VariantName`	The `::` operator is required
Variant values are auto-assigned	Sequential numbers 0, 1, 2, … (manual specification is not supported)
Comparison uses integer comparison	`==`, `!=` can be used

English | |

Testing

Ry has a built-in RSpec-style test syntax. Test files are executed using the `ry test` subcommand.

Running Tests

```
ry test              # Auto-discover and run all *.test.ry files in the project
ry test test_file.ry # Run a specific test file
```

The exit code is 0 if all tests passed, 1 if any test failed.

Auto-Discovery Mode

When `ry test` is run without arguments, it:

1. Searches for `ry.toml` to find the project root
2. Recursively discovers all `*.test.ry` files under the project root (`.git`, `build`, `node_modules` are skipped)
3. Runs each file and aggregates results

Syntax

describe / it

```
describe("description", fn():
    it("test case name", fn():
        # test body
```

```
        expect(actual_value).to_eq(expected_value)
    )
)
```

- `describe` and `it` take a description string and a **lambda argument `fn():`** as the second parameter
- `it` blocks and other statements (e.g., variable declarations) can be written inside a `describe` block
- Each `it` block is an independent test case
- `describe` / `expect` are only available with `ry test` (compile error with normal `ry` execution)

Trailing Block Syntax

Any function call (except `describe`/`it`/`mock`) can use trailing block syntax. A colon after `()` causes the indented block to be passed as a no-argument lambda in the last argument position:

```
# These are equivalent:
foo("arg"):
    bar()

foo("arg", fn():
    bar()
)
```

expect / Matchers

Matcher	Description	Supported Types
`to_eq(expected)`	Equality comparison	int, float, bool, str
`to_not_eq(expected)`	Asserts not equal	int, float, bool, str
`to_be_true()`	Asserts `true`	bool
`to_be_false()`	Asserts `false`	bool
`to_be_none()`	Asserts `None`	Option
`to_be_some()`	Asserts Option is `Some`	Option
`to_contain(val)`	Asserts container includes value	List, Set, Map, str
`to_not_contain(val)`	Asserts container does not include value	List, Set, Map, str
`to_be_greater_than(v)`	Asserts `actual > v`	int, float
`to_be_less_than(v)`	Asserts `actual < v`	int, float
`to_be_greater_than_or_eq(v)`	Asserts `actual >= v`	int, float
`to_be_less_than_or_eq(v)`	Asserts `actual <= v`	int, float
`to_have_length(n)`	Asserts length equals `n`	List, Set, Map, str
`to_be_empty()`	Asserts length is 0	List, Set, Map, str
`to_start_with(prefix)`	Asserts string starts with prefix	str
`to_end_with(suffix)`	Asserts string ends with suffix	str

Output Format

```
Calculator
  + adds numbers
  + subtracts
  - fails test (red)
    line 10: expected 3, got 2

2 passed, 1 failed
```

- `+` indicates pass (green), `-` indicates failure (red)
- On failure, the line number and expected/actual values are displayed

---

Example

```
describe("Arithmetic", fn():
    it("adds integers", fn():
        expect(1 + 2).to_eq(3)

    )
    it("compares strings", fn():
        expect("hello").to_eq("hello")

    )
    it("checks booleans", fn():
        expect(3 > 1).to_be_true()

    )
)
describe("Booleans", fn():
    it("false check", fn():
        expect(1 > 2).to_be_false()
    )
)
```

Mocking

mock(fn_name, replacement)

Replaces a function with a mock implementation for the current `it` block. The mock is automatically cleared when the `it` block ends.

```
fn fetch_data() -> str:
    return "real data"

describe("mocking", fn():
    it("replaces function", fn():
        mock(fetch_data, fn(): "fake")
        expect(fetch_data()).to_eq("fake")

    )
    it("auto-restores", fn():
        expect(fetch_data()).to_eq("real data")
    )
)
```

- The first argument is the function name (identifier, not a string)
- The second argument is a replacement lambda
- The replacement must have the same parameter types and return type as the original function
- Mocks are automatically restored at the end of each `it` block

verify(fn_name)

Returns the number of times a mocked function was called (as `int`).

```
describe("verify", fn():
    it("counts calls", fn():
        mock(fetch_data, fn(): "fake")
        fetch_data()
        fetch_data()
        expect(verify(fetch_data)).to_eq(2)
    )
)
```

Limitations

- Overloaded functions cannot be mocked
- Capture-based closures cannot be used as replacements (use plain lambdas)
- `@native fn` functions cannot be mocked

Parameterized Tests (@each)

`@each` runs the same test with multiple sets of parameters. Attach it to an `it` block with a list of tuples:

```
@each([
    (1, 2, 3),
    (0, 0, 0),
    (-1, 1, 0)
])
it("adds {0} + {1} = {2}", fn(a: int, b: int, expected: int):
    expect(a + b).to_eq(expected)
)
```

- The list must contain tuples whose arity matches the lambda parameter count
- Placeholders `{0}`, `{1}`, … in the description are replaced with the parameter values
- Each tuple generates an independent test case
- Supported parameter types: `int`, `float`, `bool`, `str`

Property-Based Tests (@property)

`@property` generates random inputs and runs the test multiple times:

```
@property(count=100)
it("addition is commutative", fn(a: int, b: int):
    expect(a + b).to_eq(b + a)
)
```

- `count=N` specifies the number of random trials (default: 100)
- On failure, the counterexample (failing inputs) is printed
- The test stops at the first failure
- Supported parameter types: `int` ([-1000, 1000]), `float` ([-1000.0, 1000.0]), `bool`, `str` (random ASCII, 0-20 chars)

Limitations

- Nesting of `describe` is not supported
- `before_each` / `after_each` are not supported

English | |

Type Reference

Type List

Type	Internal Representation	Literal Examples	Description
`int`	i64	`42`, `-7`, `0xFF`, `0b1010`	64-bit signed integer
`byte`	i8	(no dedicated literal)	Unsigned 8-bit integer (0-255). Used with type annotation `let b: byte = 42`
`float`	f64	`3.14`, `0.5`	64-bit floating-point number
`bool`	i1	`true`, `false`	Boolean value
`str`	ptr	`"hello"`, `""`, `"a\nb"`	String (immutable byte sequence on the heap)
`Unit`	void	(no return value)	Implicit return type when return type is omitted
`Option<T>`	`{ i1, T }`	`Some(42)`, `None`	A type that may or may not contain a value
`(T1, T2, ...)`	LLVM StructType (literal)	`(1, 3.14)`	Tuple type
`List<T>`	ptr (heap)	`[1, 2, 3]`	Dynamic array
`Map<K, V>`	ptr (heap)	`{"a": 1}`	Hash map
`Set<T>`	ptr (heap)	`{1, 2, 3}`	Set with no duplicates
`fn(T1, T2) -> R`	ptr (function pointer)	`fn(x: int): x * 2`	Function type
User-defined type	LLVM StructType (named)	`record Point: ...`	Struct defined with the `record` keyword
`enum`	i64 / tagged union	`Color::Red`, `Shape::Circle(3.14)`	Enumeration defined with the `enum` keyword (supports associated data)
`Error`	`{ ptr, i64 }`	`Error("msg")`, `Error("msg", 404)`	Built-in error type
`T1 \\| T2`	`{ i64, [N x i8] }`	`int \\| str`	Union type (holds one of multiple types)
Int literal	i64	`42`, `0 \\| 1`	Int literal type (value constraint)
String literal	ptr	`"N" \\| "S"`	String literal type (value constraint)
Range	i64	`1..12`, `-10..10`	Range type (inclusive integer range constraint)

Type Annotation Syntax

You can explicitly specify the type when declaring a variable. The annotation can be omitted when the type is inferrable.

```
let x: int = 42
let b: byte = 255
let f: float = 3.14
let s: str = "hello"
let b: bool = true
let opt: Option<int> = Some(10)
let t: (int, float) = (1, 3.14)
let xs: List<int> = [1, 2, 3]
let m: Map<str, int> = {"a": 1}
let s: Set<int> = {1, 2, 3}
let fn_val: fn(int) -> int = fn(x: int): x * 2
let u: int | str = 42
```

Available Type Names

Type Name	Notes
`int`	Built-in scalar type
`byte`	Built-in scalar type (unsigned 0-255)
`float`	Built-in scalar type
`bool`	Built-in scalar type
`str`	Built-in string type
`Unit`	Return type of functions that return no value
`Option<T>`	Generic type (T is any type)
`(T1, T2, ...)`	Tuple type (arbitrary number and combination of element types)
`List<T>`	Generic dynamic array type
`Map<K, V>`	Generic hash map type
`Set<T>`	Generic set type
`fn(T1, ...) -> R`	Function type
`Error`	Built-in error type (`message: str`, `code: int`)
`T1 | T2 | ...`	Union type (one of multiple types separated by `|`)
User-defined type name	Type declared with the `record` or `enum` keyword

Type Aliases

The `type` keyword creates a new name for an existing type. The alias is fully interchangeable with the original type.

```
type Meters = float
type StringList = List<str>

let d: Meters = 3.14
let names: StringList = ["Alice", "Bob"]
```

> **Naming convention**: Type alias names must use PascalCase (e.g., `Meters`, `StringList`). The compiler enforces this convention.

Type aliases also work with function types, literal types, and range types:

```
type Callback = fn(int, int) -> int

let add: Callback = fn(a: int, b: int): a + b
print(add(3, 4))    # 7
```

```
type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

let m: Month = 6
let d: Direction = "N"
let n: Digit = 5
```

Literal Types

A literal type restricts a variable to specific constant values. The compiler checks these constraints at compile time for constant values, and emits runtime checks for dynamic values.

Int Literal Type

```
let x: 42 = 42           # single literal type
let y: 0 | 1 = 0         # union of int literals
var z: 0 | 1 = 0
z = 1                     # OK
# z = 2                   # compile error (constant) or runtime error (dynamic)
```

String Literal Type

```
let dir: "N" | "S" | "E" | "W" = "N"
# let bad: "N" | "S" = "X"    # compile error
```

Constraint Checking

- **Compile time**: If the assigned value is a constant (`ConstantInt` or string literal), the constraint is checked at compile time and a compile error is raised on violation.
- **Runtime**: If the value is dynamic (e.g., from a function call), the constraint is checked at runtime and the program exits with an error on violation.

Range Type

A range type constrains an integer variable to a contiguous range of values (inclusive on both ends).

```
let month: 1..12 = 6        # OK
# let bad: 1..12 = 0        # compile error: out of range
# let bad: 1..12 = 13       # compile error: out of range

let t: -10..10 = -5         # negative ranges are supported
```

With var (Runtime Check)

```
var x: 1..12 = 6
x = 12                       # OK
# x = dynamic_value()        # runtime check: exits if out of range
```

In Function Parameters

```
fn set_month(m: 1..12) -> int:
    return m
```

```
set_month(6)                # OK
# set_month(13)             # compile error (constant argument)
```

---

none Keyword and Option Type Shorthand

The `none` keyword represents the absence of a value for Option types, equivalent to `None`.

The `T?` syntax is a shorthand for `Option<T>`.

```
let x: int? = 42        # equivalent to Option<int>
let y: int? = none       # equivalent to None

fn find(xs: List<int>, val: int) -> int?:
    for x in xs:
        if x == val:
            return Some(x)
    return none
```

F-String (String Interpolation)

String interpolation with the `f"..."` syntax. Expressions inside `{}` are evaluated and converted to strings.

```
let name = "world"
print(f"Hello {name}")      # Hello world

let a = 1
let b = 2
print(f"{a} + {b} = {a + b}")    # 1 + 2 = 3
```

Supported Types in Interpolation

Any expression that evaluates to `int`, `float`, `bool`, or `str` can be used inside `{}`.

Escape Sequences

Sequence	Output
`{{`	`{` (literal brace)
`}}`	`}` (literal brace)
`\n \t \\ \"`	Same as regular strings

```
print(f"{{braces}}")    # {braces}
```

Type Casting (as)

Explicit type conversion using the `as` keyword.

```
let x = 42 as float      # 42.0
let y = 3.14 as int       # 3
let z = 1 as bool         # true
let s = 42 as str          # "42"
let b = 255 as byte        # byte value 255
```

Supported Casts

From	To	Behavior
`int`	`float`	`SIToFP`
`float`	`int`	Truncation (`FPToSI`)
`int`	`bool`	`0` -> `false`, non-zero -> `true`
`bool`	`int`	`false` -> `0`, `true` -> `1`
`int` / `float` / `bool`	`str`	String representation
`int`	`byte`	Truncation (lower 8 bits)
`byte`	`int`	Zero extension

Unsupported casts (e.g. `str as int`) cause a compile error. Use `to_int()` / `to_float()` for string-to-number conversions.

Enum with Associated Data (ADT)

Enum variants can carry associated data by specifying types in parentheses after the variant name. Variants without parentheses remain simple tags.

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

Constructor

Use the `EnumName::Variant(value)` syntax to construct a variant with data.

```
let c = Shape::Circle(3.14)
let r = Shape::Rectangle(4.0, 5.0)
let p = Shape::Point
```

Pattern Matching with Binding

Use `case EnumName::Variant(binding):` to extract the associated data.

```
match c:
    case Shape::Circle(r):
        print(r)            # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")
```

Internal Representation

An ADT enum is stored as a tagged union: `{ i64 tag, [N x i8] data }` where `N` is sized to fit the largest variant's payload.

Generic Enum

An enum can have type parameters using angle-bracket syntax `<T>`. This allows the same enum shape to hold different payload types.

```
enum MyOption<T>:
    MySome(T)
    MyNone
```

Usage

Instantiate by providing a concrete type argument. The type argument is required when the compiler cannot infer it.

```
let a = MyOption<int>::MySome(42)
let b = MyOption<int>::MyNone

match a:
    case MyOption::MySome(v):
        print(v)      # 42
    case MyOption::MyNone:
        print("none")
```

Error Type

A built-in type for error handling. `Error` has two fields: `message` (str) and `code` (int).

```
let e = Error("something went wrong")       # code defaults to 0
let e2 = Error("not found", 404)            # explicit code

print(e.message)   # something went wrong
print(e2.code)     # 404
print(e2)          # Error: not found (code: 404)
```

Error Handling Convention

Functions that can fail return a `(T, Error?)` tuple:

```
fn divide(a: int, b: int) -> (int, Error?):
    if b == 0:
        return (0, Some(Error("division by zero")))
    return (a // b, none)

let val, err = divide(10, 2)
match err:
    case Some(e):
        print(e.message)
    case None:
        print(val)          # 5
```

`!!` Operator (Error Propagation)

The `!!` postfix operator extracts the value from a `(T, Error?)` tuple. If the error is present, it is propagated to the enclosing function.

```
fn read_file(path: str) -> (str, Error?):
    if path == "":
        return ("", Some(Error("empty path")))
    return ("content", none)

fn process() -> (str, Error?):
```

```
    let data = read_file("test.txt")!!    # propagates error if any
    return (data, none)
```

The enclosing function must also return `(X, Error?)` for `!!` to work.

Internal Representation

`Error` is represented as `{ ptr message, i64 code }`.

Union Type

You can declare a variable that may hold one of multiple types using `|`.

```
let x: int | str = 42
x = "hello"      # Reassignment is allowed (any type in the union)
print(x)         # hello
```

Usage in Function Parameters and Return Types

```
fn show(x: int | str) -> int:
    print(x)
    return 0

fn get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

Internal Representation

A union type is represented as `{ i64 tag, [N x i8] data }`. The `tag` indicates the index of each component type (sorted alphabetically), and `data` is a byte array sized to the largest component type.

Constraints

- Assigning a type not included in the union causes a compile error
- `int | str` and `str | int` are the same type (normalized)
- When printing a union value with `print()`, the value is displayed using the appropriate type based on the runtime tag

Type Rules (Type Conversion in Operations)

Operation	Left	Right	Result Type	Notes
`+ - *`	int	int	int	
`+ - *`	byte	byte or int	int	byte is ZExt-promoted to int during operations
`+ - *`	float or int	float or int (one is float)	float	Implicit float promotion
`/`	any numeric	any numeric	float	Always float
`//`	any numeric	any numeric	int	Float input is truncated
`**`	any numeric	any numeric	float	Uses libm `pow`
`%`	int	int	int	

Operation	Left	Right	Result Type	Notes
`%`	float or int	float or int (one is float)	float	
`+`	str	str	str	String concatenation
`== != < <= > >=`	str	str	bool	Lexicographic comparison
`== != < <= > >=`	numeric or bool	numeric or bool	bool	
`in`	any	Set	bool	Whether the element is in the set
`& | ^ ~ << >>`	int	int	int	Error for float

Escape Sequences (in str Literals)

Sequence	Meaning
`\n`	Newline
`\t`	Tab
`\\`	Backslash
`\"`	Double quote
`\0`	Null character

Type Safety Constraints

- **No implicit type conversions** – Mixing `int` and `float` triggers float promotion, but no other implicit conversions exist. `byte` is automatically promoted to `int` during operations (ZExt). Narrowing conversion from an `int` literal to `byte` is only allowed with a type annotation `let b: byte = 42`.
- **Variable types are fixed at declaration** – A variable declared as `int` cannot be reassigned a `float` value.
- **Bitwise operations are for `int` only** – Applying bitwise operations to `float` or `bool` causes a compile error.
- **Non-`bool` types can be used in conditions** – `if` conditions accept `int` (0 = false, non-zero = true) and other types besides `bool`.